

Models Rmd

2026-06-24

Models

- Package Installation

```
library("rethinking")
library(dplyr)
library(dagitty)
```

Causal effects of the DAG

```
kdag<-dagitty("dag {
  S-> K <- T
  T -> S
  T -> SL -> K
}")
adjustmentSets(kdag, exposure="SL", outcome = "K")# This shows the backdoor path and how to stratify
## { T }
```

Synthetic Data

- create a synthetic data with assumptions. See overview.pdf to check the data relationship.

```
# Function of SL by Y
# This is arbitrary made by me, immitating the research by Susumu Tanabe and Rei Nakashima in 2026
# Y is between 0 and 1, which represents 16,000 years ago to 2,400 years ago.
myFunc <- function(x) {
  case_when(
    0 <= x & x <= (-9+ 16)/(13.6) ~ -(x * (-13.6)) * 60 / 7 - 80,
    (-9.000+ 16)/(13.6) < x & x <= (-7.500 + 16)/(13.6) ~ -(x * (-13.6) + 7) * 46 / 3 - 20,
    (-7.500+ 16)/(13.6) < x & x <= (-4.3+ 16)/(13.6) ~ 3,
    (-4.3 + 16)/(13.6) < x & x <= (-3.000+ 16)/(13.6) ~ -(x * (-13.6) + 11.7) * (-60) / 13 + 3,
    (-3.000+ 16)/(13.6) < x & x <= (-2.400+ 16)/(13.6) ~ -(x * (-13.6) + 13) * 5 / 3 - 1,
    TRUE ~ 1
  )
}
curve(myFunc(x), col = 2, main = "Relative Sea Level and Year")

# Initialize Y
Y<- runif(200, 0, 1)

# T and SL caused by Y
T<- NULL
SL<-NULL
for (i in 1:length(Y)) {
```

```

T[i]<-rpois(1, Y[i]* 25)
if(T[i] < 1) {T[i]<- 1}
else if(T[i] > 25) {T[i]<- 25} # Make sure T is between 1 and 25
SL[i]<- rnorm(1, myFunc(Y[i]), 5)
}

SL <- (SL - min(SL)) / (max(SL) - min(SL)) # to estimate easily, make it between 0 and 1

# Check the data
plot(T, SL, main = "Synthetic Data Plot(T and SL)")
plot(Y, T, main = "Synthetic Data Plot(Y and T)")
plot(Y, SL, main = "Synthetic Data Plot(Y and SL)")

# initialize dS with random numbers
dS<-rbeta(length(T), 1.2,1) * 200
# S is affected by Y and dS (omit T as T and S are correlated by Y)
S<-NULL
for(i in 1:length(T))S[i] <- rnorm(1, 5*Y[i] -dS[i]/50, 0.1)/25 + 0.3

# Check the data
range(S) # try to make less than 0.5

## [1] 0.1476331 0.4808579

plot(density(S), main = "Density of Synthetic Data of S")

plot(dS, S, main = "Synthetic Data Plot(dS and S) \n with different groups")
legend(x = 0, y = 0.18, legend="red: T = 2 blue: T = 7 green: T = 13")
points(dS[T==2], S[T==2], col = 2, lwd =4)
points(dS[T==13], S[T==13], col = 3, lwd =4)
points(dS[T==7], S[T==7], col = 4, lwd =4)
plot(T, S, main = "Synthetic Data Plot(T and S)")
plot(Y, S, main = "Synthetic Data Plot(Y and S)")
plot(T, dS, main = "Synthetic Data Plot(T and dS)\n No correlation expected.") # no correlation
plot(SL, S, main = "Synthetic Data Plot(SL and S)\n auto-correlation")

# Initialize dK with random numbers
dK<-rbeta(length(T), 1.4,1)*400
# K is affected by SL, Y, S, dK
K<-NULL
for(i in 1:length(T))K[i] <- rnorm(1, 1 * SL[i] + 1 * Y[i] + 1 * S[i]- dK[i]/250, 0.2)/10 + 0.2

# Check the data
range(K) # try to make less than 0.5

## [1] 0.0644508 0.4006054

plot(density(K), main = "Density of Synthetic Data of K")
plot(dK, K, main = "Synthetic Data Plot(dK and K)")
plot(SL, K, main = "Synthetic Data Plot(SL and K)")
plot(Y, K, main = "Synthetic Data Plot(Y and K)")
plot(S, K, main = "Synthetic Data Plot(S and K)")
plot(dS, K, main = "Synthetic Data Plot(dS and K)\n No correlation expected.") # no correlation

# Standardize data without SL, S, K (those are ratio)

```

```
d<-list(T=T, dS=standardize(dS), S=S, K=K, dK=standardize(dK))
```

```
# check the data
```

```
plot(density(d$K), main = "Density of Synthetic Data of d$K")
```

```
plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
```

```
plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
```

```
plot(Y, d$K, main = "Synthetic Data Plot(Y and d$K)")
```

```
plot(d$T, d$K, main = "Synthetic Data Plot(d$T and d$K)")
```

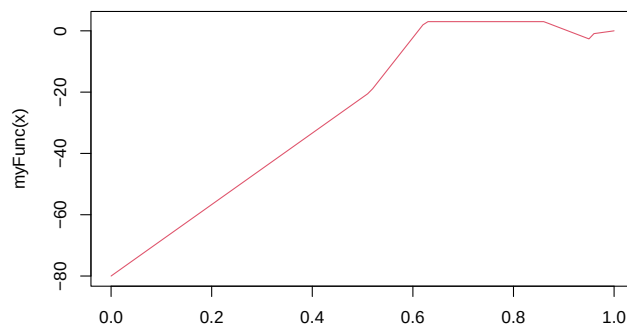
```
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
```

```
plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")
```

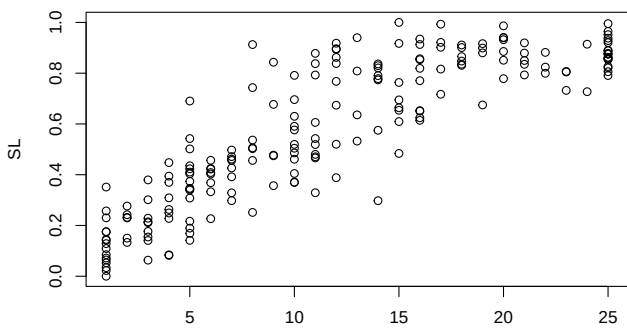
```
plot(Y, d$S, main = "Synthetic Data Plot(Y and d$S)")
```

```
plot(d$dS, d$K, main = "Synthetic Data Plot(d$dS and d$K)\n No correlation expected.") # no correlation
```

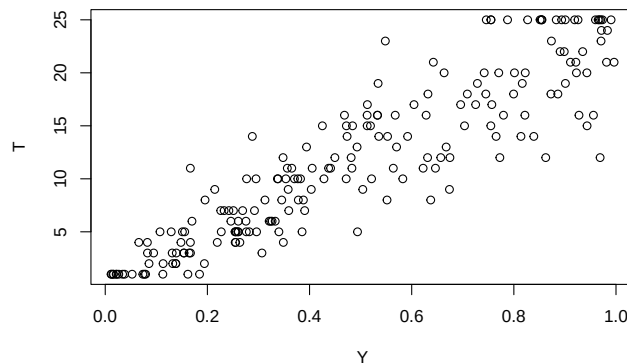
Relative Sea Level and Year



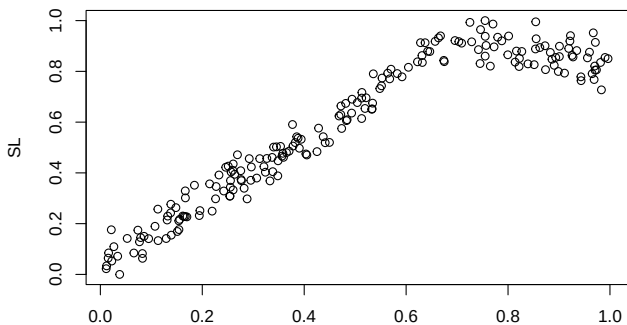
Synthetic Data Plot(T and SL)



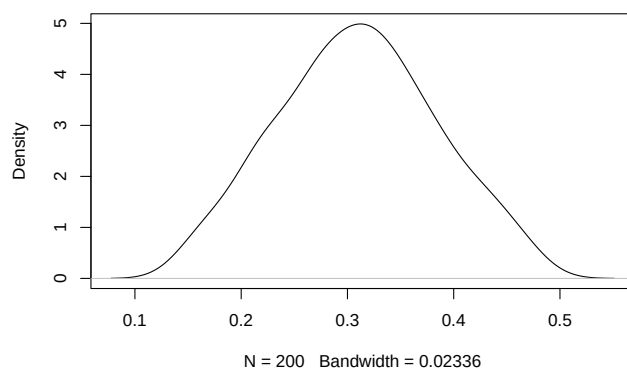
Synthetic Data Plot(Y and T)



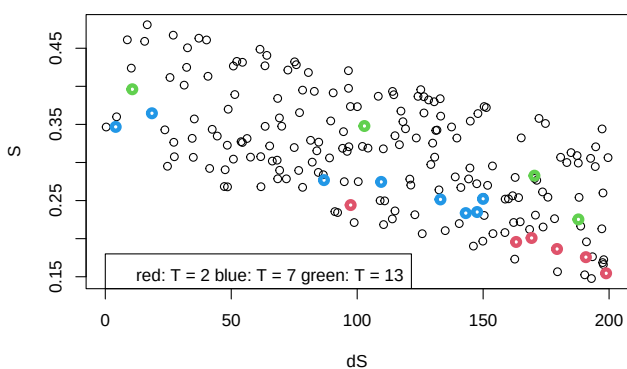
Synthetic Data Plot(Y and SL)



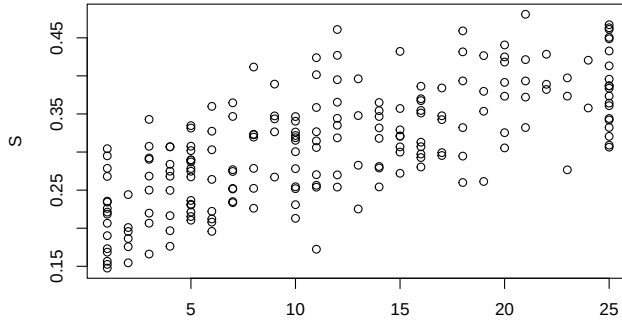
Density of Synthetic Data of S



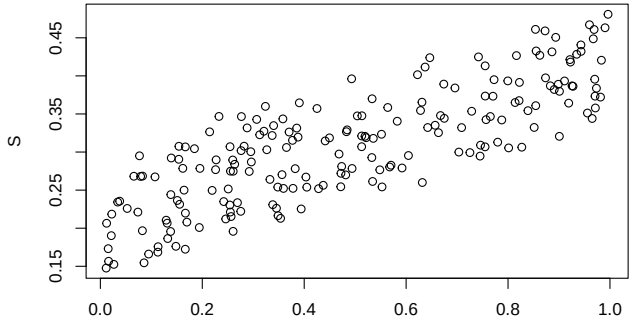
Synthetic Data Plot(dS and S)
with different groups



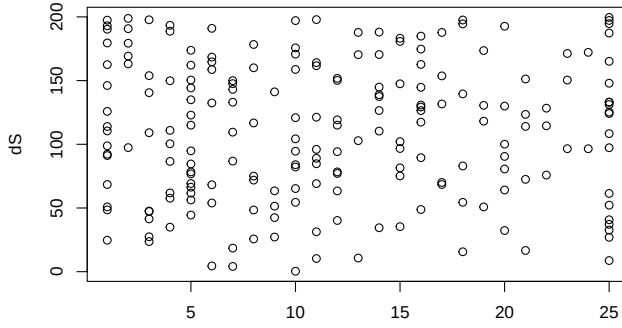
Synthetic Data Plot(T and S)



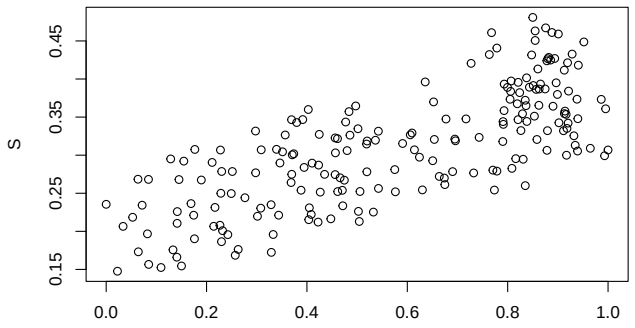
Synthetic Data Plot(Y and S)



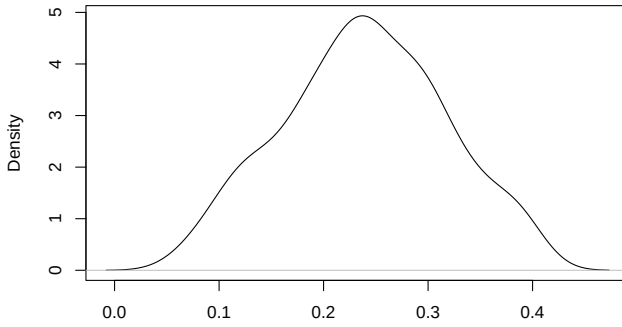
Synthetic Data Plot(T and dS)
No correlation expected.



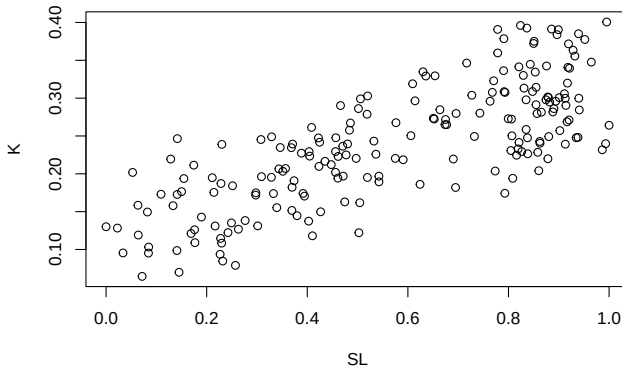
Synthetic Data Plot(SL and S)
auto-correlation



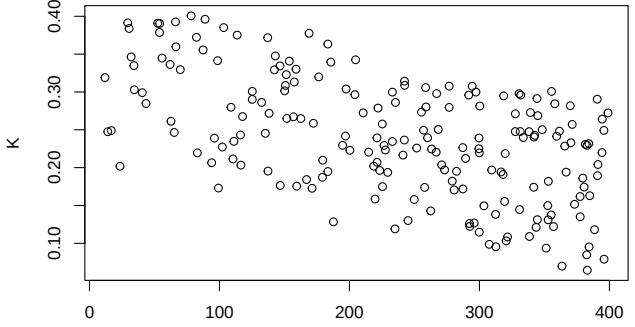
Density of Synthetic Data of K



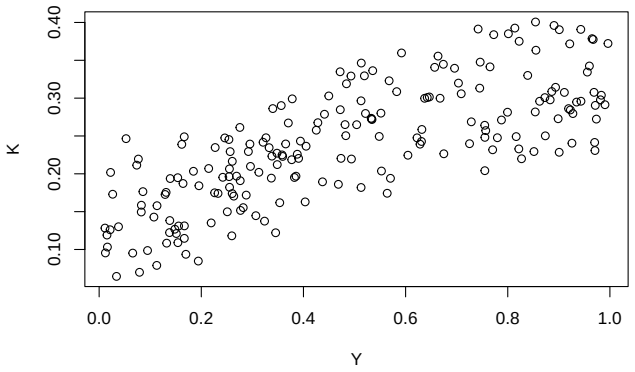
N = 200 Bandwidth = 0.0242
Synthetic Data Plot(SL and K)

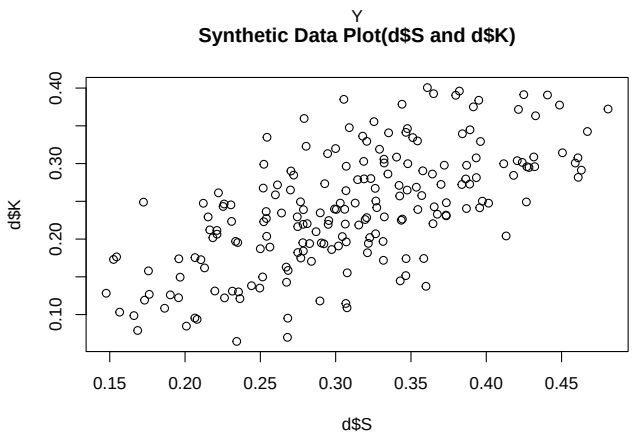
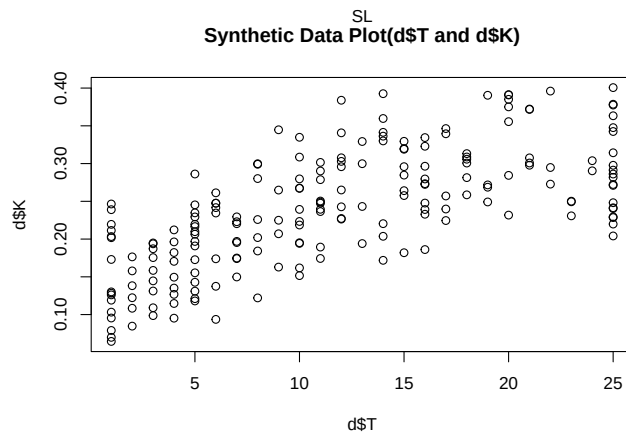
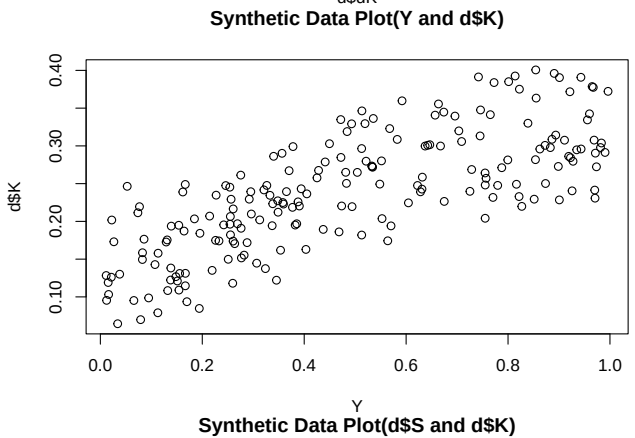
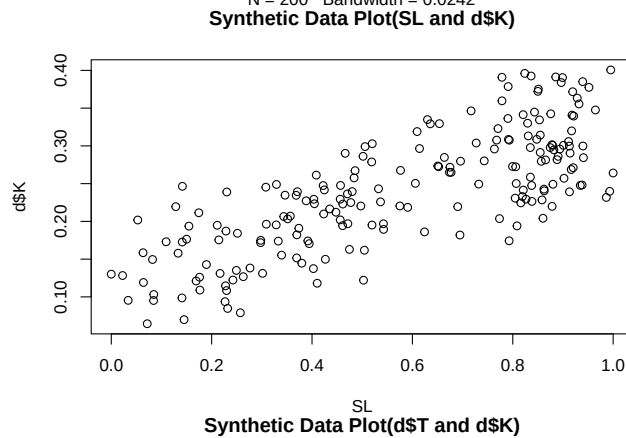
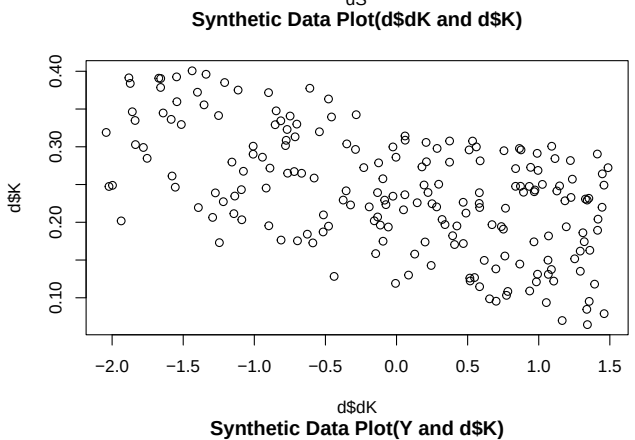
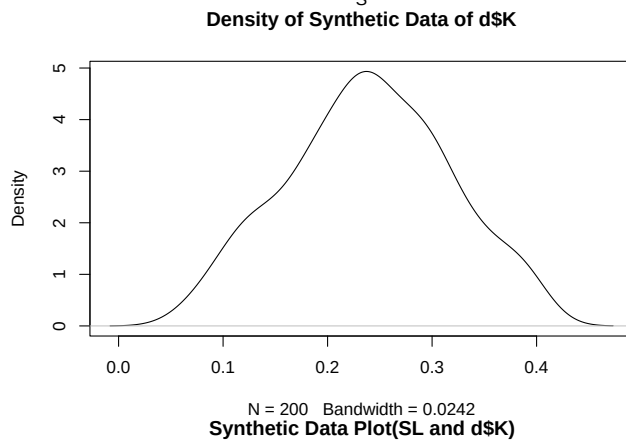
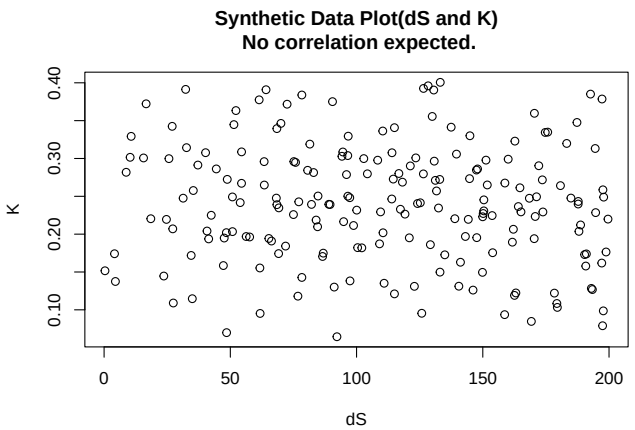
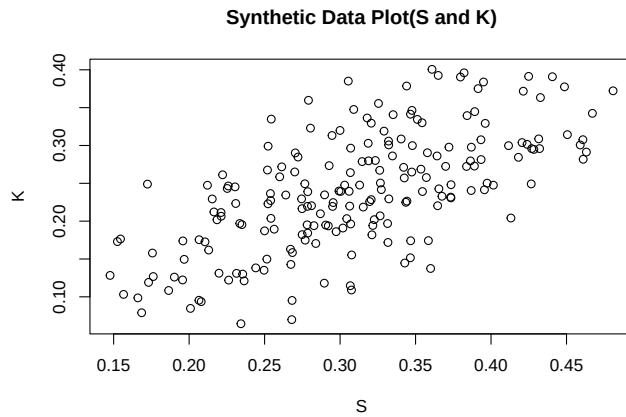


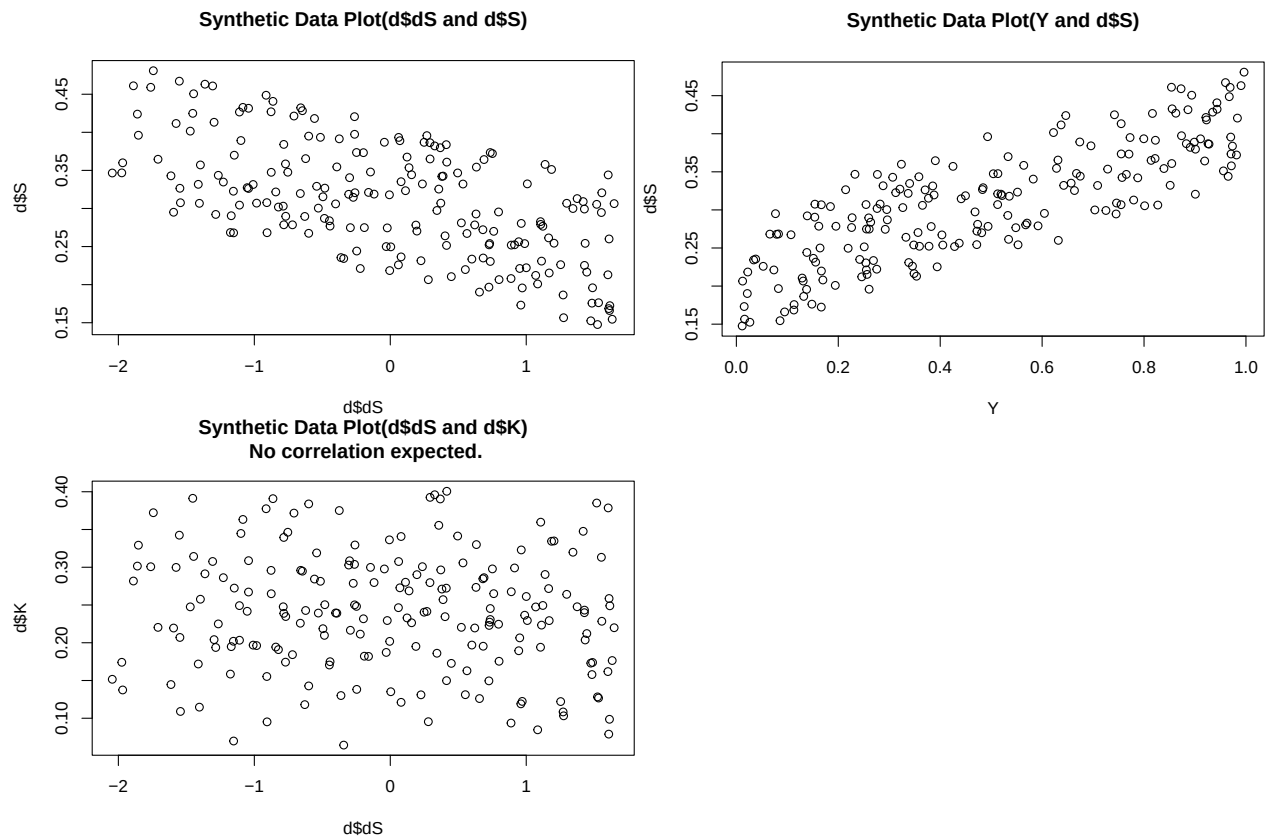
Synthetic Data Plot(dK and K)



Synthetic Data Plot(Y and K)



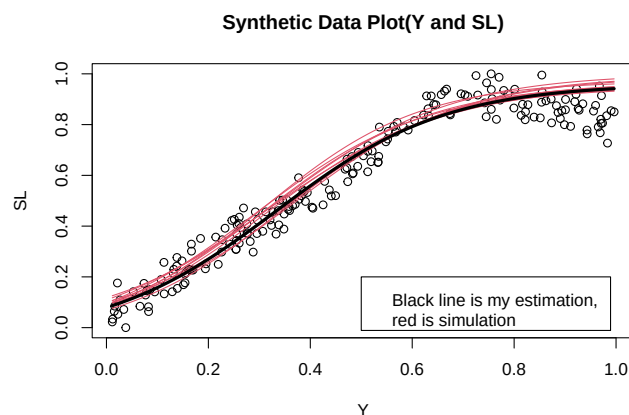




Prior Prediction

- Now I want to create a prior and statistical model.
- As in the overview, the Y and SL are hidden parameter so how to initialize is important
- This time, to make it simple, I use logit function to immitate the *myFunction*

```
plot(Y, SL, main = "Synthetic Data Plot(Y and SL)")
# Simulation
for(i in 1:10)curve(inv_logit(rnorm(1, 6, 0.2)*x + rnorm(1, -2, 0.2)) - rnorm(1, 0.04, 0.02), add =TRUE)
curve(inv_logit(6*x - 2) - 0.04, add =TRUE, lwd = 3)
legend(0.5, 0.2, legend="Black line is my estimation, \nred is simulation")
```



Model 1

- Also see overview.pdf to know statistical model and prior
- This model stratifies all parameters by T.
- There are a few messages to warning my priors. Since it only shows up during first iteration of warmup, we can safely ignore it.
- I just omit it for m1.2 and 1.3.

```
# Model 1
m1<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_Y[T] * Y[i] + bK_dK * dK,

    # We do not know anything about this but think similar for all group.
    # So use partial pooling to regularize the estimate.
    # mean should start at 0 and has small variation.

    # In this case (m1), we do not include correlation.
    # This is because multilevel models of S and SL includes the information of Y
    # But we have to regularize.
    vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
    vector [25]: bK_S <- bK_S_bar + sigma_bK_S * z_bK_S[T],
    vector [25]: bK_SL <- bK_SL_bar + sigma_bK_SL * z_bK_SL[T],
    vector [25]: bK_Y <- bK_Y_bar + sigma_bK_Y * z_bK_Y[T],
    c(aK_bar, bK_S_bar, bK_SL_bar, bK_Y_bar) ~ normal(0,0.3),
    c(sigma_aK, sigma_bK_S, sigma_bK_SL, sigma_bK_Y) ~ exponential(6), # want low variation among g
    z_aK[T] ~ normal(0,0.1),
    z_bK_S[T] ~ normal(0,0.1),
    z_bK_SL[T] ~ normal(0,0.1),
    z_bK_Y[T] ~ normal(0,0.1),

    # don't need regularization as dK is not affected by T
    # We are hoping/believing b_dK is negative, but this is not prior.
    bK_dK ~ normal(0, 0.3),
    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS[T] + bS_Y[T] * Y[i] + bS_dS * dS,

    # I am hoping/believing b_dS is negative and b_Y_S is positive
    # partial pooling for the same reason for a2 and b_Y_S
    vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
    vector[25]: bS_Y<-bS_Y_bar + sigmabS_Y * zbS_Y[T],
    c(aS_bar, bS_Y_bar) ~ normal(0,0.3),
    c(sigma_aS, sigmabS_Y) ~ exponential(6),
    z_aS[T] ~ normal(0,0.1),
    zbS_Y[T] ~ normal(0,0.1),

    # don't need regularization
    bS_dS ~ normal(0,0.3),
    sigma_S ~ exponential(1),
```

```

# Model SL by Y
# The log function and prior was referred to the previous research of Sea Level Change over time
# This function is abstract so I will have to actually stan code to provide accurate estimates
# ulam code in r does not allow us to hard code the conditional function.
# vector[200]: SL ~ normal(muSL + aSL2, sigmaSL),
vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

# Model T, This should be hard coded based on the previous research
T ~ poisson(Y * 25),

# Model T by Y
vector[200]:Y ~ normal(0.5, 0.3)

), dat = d, constraints = list(
  SL = "lower=0,upper=1", # SL is between 0 and 1
  Y = "lower=0,upper=1" # Y is between 0 and 1
  #, T = "lower=1,upper=25" #this is unnecessary since T is provided in the dat

), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE
)

```

```

## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write
## 205 | void write_array(stan::rng_t& rng, std::vector<double>& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpS4aHLM/model-34c46ba33490.hpp:1843:3: note: by 'void ulam_cm
## 1843 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write
## 136 | void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpS4aHLM/model-34c46ba33490.hpp:1843:3: note: by 'void ulam_cm
## 1843 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_crtp
## 96 | inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
## | ~~~~~
## C:/Use
## rs/nobut/AppData/Local/Temp/RtmpS4aHLM/model-34c46ba33490.hpp:1880:3: note: by 'T_ ulam_cmds
## 1880 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::log
## 154 | inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpS4aHLM/model-34c46ba33490.hpp:1880:3: note: by 'T_ ulam_cmds
## 1880 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log
## 91 | inline double log_prob(Eigen::VectorXd& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpS4aHLM/model-34c46ba33490.hpp:1880:3: note: by 'T_ ulam_cmds

```



```

## const'
## 1880 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      |   ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_crtp::
## 159 |   inline math::var log_prob(std::vector<math::var>& theta,
##      |   ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpS4aHLM/model-34c46ba33490.hpp:1880:3: note:   by 'T_ ulam_cmds
## 1880 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      |   ~~~~~

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified
## Chain 1
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified
## Chain 2
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified
## Chain 3
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppData
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified
## Chain 4
## Chain 1 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:  200 / 4000 [ 5%] (Warmup)

```

```

## Chain 2 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 4 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 2 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 3 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 1 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 4 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 2 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 3 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 1 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 4 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 2 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 3 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 1 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 2 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 3 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 4 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 1 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 3 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)

```

```

## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)

```

```

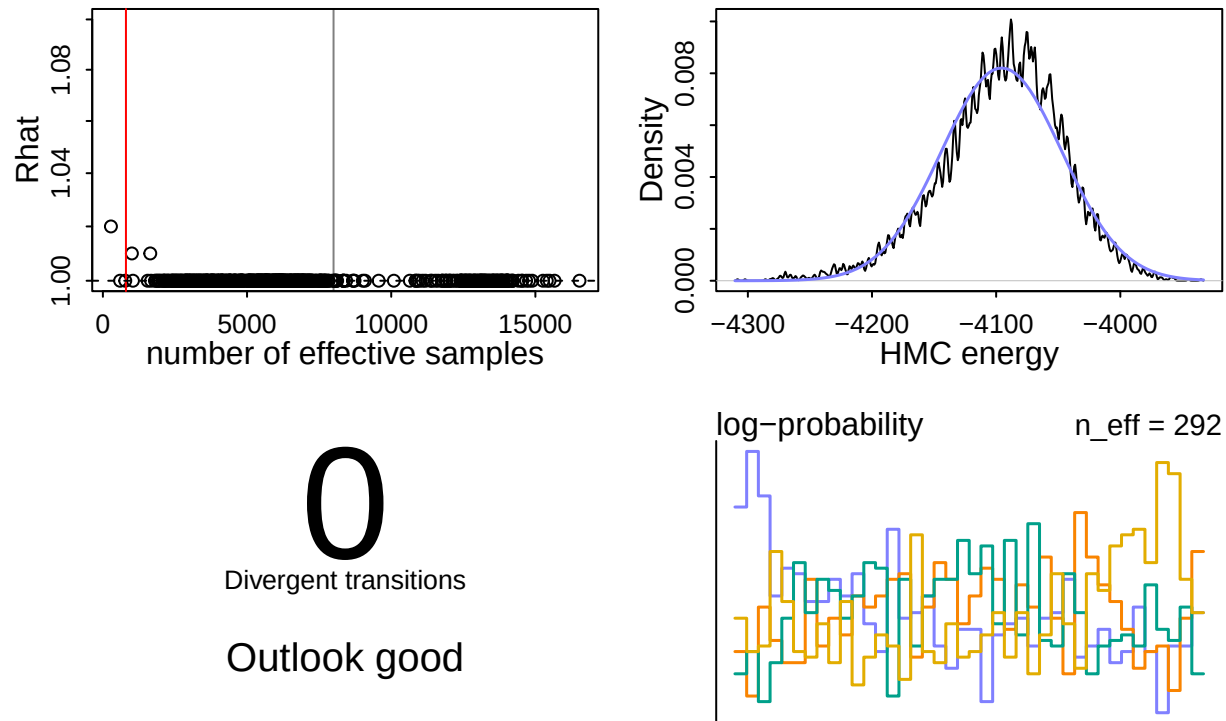
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 44.3 seconds.
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 51.4 seconds.
## Chain 3 finished in 51.4 seconds.
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 52.1 seconds.

```

```
##
## All 4 chains finished successfully.
## Mean chain execution time: 49.8 seconds.
## Total execution time: 52.2 seconds.

## Warning: 4 of 4 chains had an E-BFMI less than 0.3.
## See https://mc-stan.org/misc/warnings for details.
```

```
dashboard(m1)
```



- This has so much divergent and less effective sampling.
- This would be due to the large number of parameters, including hidden estimated ones (SL and Y). I will reduce parameters to reliably interpret the result.
 - First one would be Y as S and SL are more important for the model.
 - * by the way, SL and Y in model K change the efficiency in the highest rate.

```
# Model 1.2
m1.2<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_dK * dK,
    vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
    vector [25]: bK_S <- bK_S_bar + sigma_bK_S * z_bK_S[T],
    vector [25]: bK_SL <- bK_SL_bar + sigma_bK_SL * z_bK_SL[T],
    c(aK_bar, bK_S_bar, bK_SL_bar) ~ normal(0,0.3),
    c(sigma_aK, sigma_bK_S, sigma_bK_SL) ~ exponential(6),
    z_aK[T] ~ normal(0,0.1),
    z_bK_S[T] ~ normal(0,0.1),
    z_bK_SL[T] ~ normal(0,0.1),
    bK_dK ~ normal(0, 0.3),
    U ~ exponential(1),
```

```

# Model S
S ~ normal(mu_S, sigma_S),
logit(mu_S) <- aS[T] + bS_Y[T] * Y[i] + bS_dS * dS ,
vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
vector[25]: bS_Y<-bS_Y_bar + sigmabS_Y * zbS_Y[T],
c(aS_bar, bS_Y_bar) ~ normal(0,0.3),
c(sigma_aS, sigmabS_Y) ~ exponential(6),
z_aS[T] ~ normal(0,0.1),
zbS_Y[T] ~ normal(0,0.1),
bS_dS ~ normal(0,0.3),
sigma_S ~ exponential(1),

# Model SL by Y
vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

# Model T, This should be hard coded based on the previous research
T ~ poisson(Y * 25),

# Model T by Y
vector[200]:Y ~ normal( 0.5, 0.2)

), dat = d, constraints = list(
  SL = "lower=0,upper=1"
  , Y = "lower=0,upper=1"
), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE
)

```

```

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 1 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:  300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:  300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration:  300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:  300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration:  400 / 4000 [10%] (Warmup)
## Chain 1 Iteration:  400 / 4000 [10%] (Warmup)
## Chain 2 Iteration:  400 / 4000 [10%] (Warmup)
## Chain 4 Iteration:  400 / 4000 [10%] (Warmup)
## Chain 1 Iteration:  500 / 4000 [12%] (Warmup)
## Chain 2 Iteration:  500 / 4000 [12%] (Warmup)
## Chain 3 Iteration:  500 / 4000 [12%] (Warmup)

```

```

## Chain 1 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 4 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 2 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 3 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 4 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 1 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 3 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 2 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 4 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 1 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 3 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)

```

```

## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)

```



```

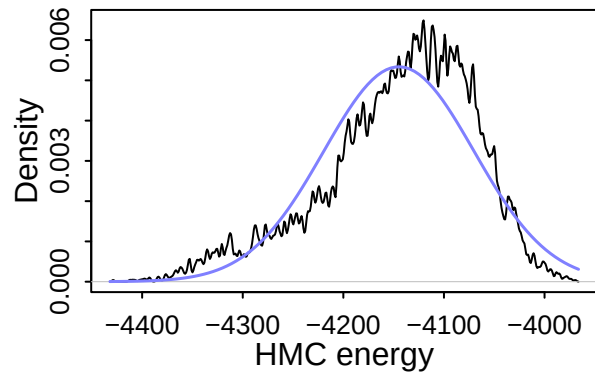
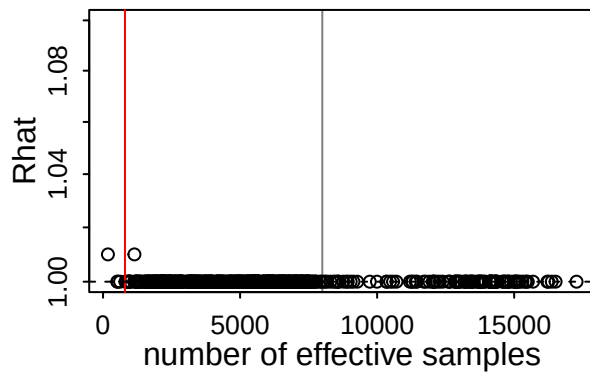
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 45.9 seconds.
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 48.5 seconds.
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 finished in 48.8 seconds.
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 49.8 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 48.2 seconds.
## Total execution time: 49.9 seconds.

```

```

dashboard(m1.2)

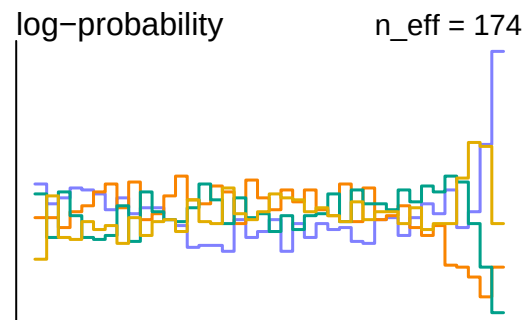
```



44

Divergent transitions

Check yourself before
you wreck yourself



This is still bad model as the sampling efficiency is very small number

```
# Model 1.3
m1.3 <- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_dK * dK,

    vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
    vector [25]: bK_S <- bK_S_bar + sigma_bK_S * z_bK_S[T],
    vector [25]: bK_SL <- bK_SL_bar + sigma_bK_SL * z_bK_SL[T],
    c(aK_bar, bK_S_bar, bK_SL_bar) ~ normal(0,0.3),
    c(sigma_aK, sigma_bK_S, sigma_bK_SL) ~ exponential(6),
    z_aK[T] ~ normal(0,0.1),
    z_bK_S[T] ~ normal(0,0.1),
    z_bK_SL[T] ~ normal(0,0.1),
    bK_dK ~ normal(0, 0.3),
    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS[T] + bS_dS * dS,
    vector[25]: aS <- aS_bar + sigma_aS * z_aS[T],
    aS_bar ~ normal(0,0.3),
    sigma_aS ~ exponential(6),
    z_aS[T] ~ normal(0,0.1),
    bS_dS ~ normal(0,0.3),
    sigma_S ~ exponential(1),

    # Model SL by Y
```

```

        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),
        # Model T, This should be hard coded based on the previous research
        T ~ poisson(Y * 25),

        # Model T by Y
        vector[200]:Y ~ normal( 0.5, 0.2)

    ), dat = d, constraints = list(
        SL      = "lower=0,upper=1",
        Y       = "lower=0,upper=1"
    ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE
)

```

Running MCMC with 4 parallel chains, with 1 thread(s) per chain...

```

##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 1 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 1 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 2 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 1 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 2 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 4 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 3 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 2 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 4 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 3 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 1 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 4 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 2 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 3 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 1 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 4 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 2 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 3 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 1 Iteration:   900 / 4000 [22%] (Warmup)

```

```

## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)

```

```

## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)

```

```

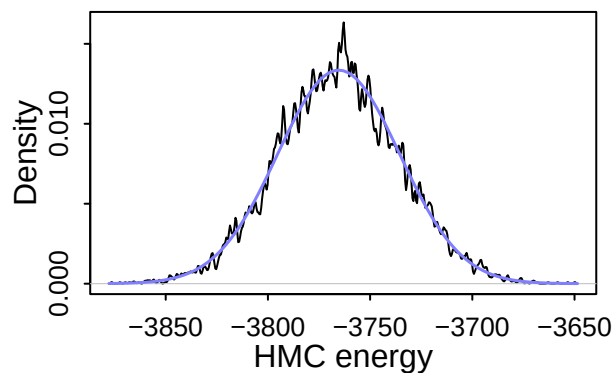
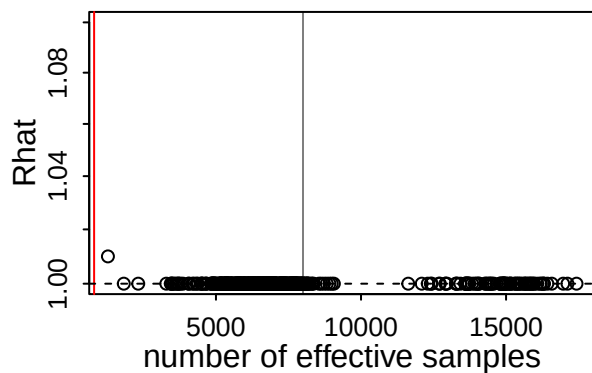
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 25.9 seconds.
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 26.5 seconds.
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 26.9 seconds.
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 33.4 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 28.2 seconds.
## Total execution time: 33.5 seconds.

```

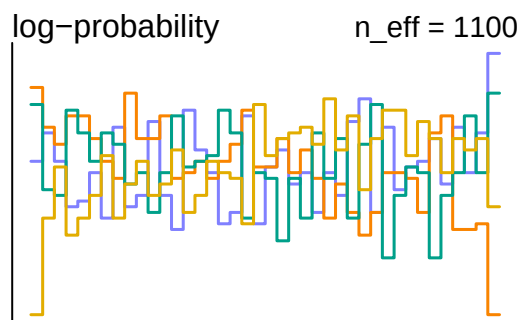
```

dashboard(m1.3)

```



0
Divergent transitions
Outlook good



```
precis(m1.3, depth=1)
```

	mean	sd	5.5%	94.5%	rhat
## bK_SL_bar	1.06361633	0.067799475	0.955944836	1.17130427	1.0026349
## bK_S_bar	1.20345311	0.177961871	0.918319207	1.48803592	1.0011826
## aK_bar	-2.20301616	0.049395181	-2.280494983	-2.12294646	1.0002090
## sigma_bK_SL	0.10015562	0.089595178	0.006186170	0.26770901	1.0001072
## sigma_bK_S	0.13809294	0.129039301	0.008630988	0.38685574	0.9999118
## sigma_aK	0.09018798	0.078737483	0.006517981	0.23966565	1.0003385
## bK_dK	-0.23570288	0.011336440	-0.253907806	-0.21764301	1.0008066
## U	0.01976285	0.002166009	0.016454531	0.02333494	1.0067085
## aS_bar	-0.75615171	0.044468560	-0.825883128	-0.68486863	1.0011475
## sigma_aS	1.68113267	0.248241086	1.327657327	2.10623466	1.0006502
## bS_dS	-0.22762842	0.013407676	-0.248904056	-0.20613413	1.0003976
## sigma_S	0.03876262	0.002048376	0.035629688	0.04210393	1.0009436
## ess_bulk					
## bK_SL_bar	1822.157				
## bK_S_bar	3557.918				
## aK_bar	3722.982				
## sigma_bK_SL	5310.161				
## sigma_bK_S	7559.517				
## sigma_aK	4638.548				
## bK_dK	4051.632				
## U	1276.600				
## aS_bar	2320.881				
## sigma_aS	3463.491				
## bS_dS	14444.497				
## sigma_S	12091.889				

- This is very high eff and no divergent, as I can see in my synthetic data (I do not know yours,,). This is great. This model is reliably sampled to be used for analysis!

Posterior Check

```
post <- extract.samples(m1.3)
```

Hidden parameters

- The hidden parameters are estimated by the model constraints and distributions.
- It is expected that the difference is small and has similar trend as the original data.

```
# Posterior Y changes
plot(1:200, Y, main = "Difference between original (black) and simulated Y (red)")
points(1:200, post$Y[1,], col = 2)
for(i in 1 : 200) lines(c(i, i), c(post$Y[1,i], Y[i]), col = 2)

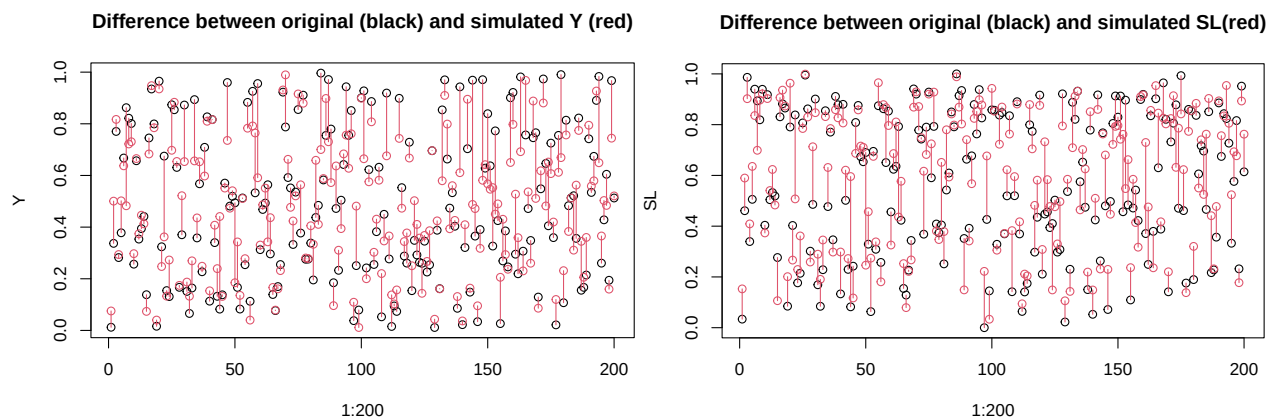
plot(1:200, SL, main = "Difference between original (black) and simulated SL(red)")
points(1:200, post$SL[1,], col = 2)
for(i in 1 : 200) lines(c(i, i), c(post$SL[1,i], SL[i]), col = 2)

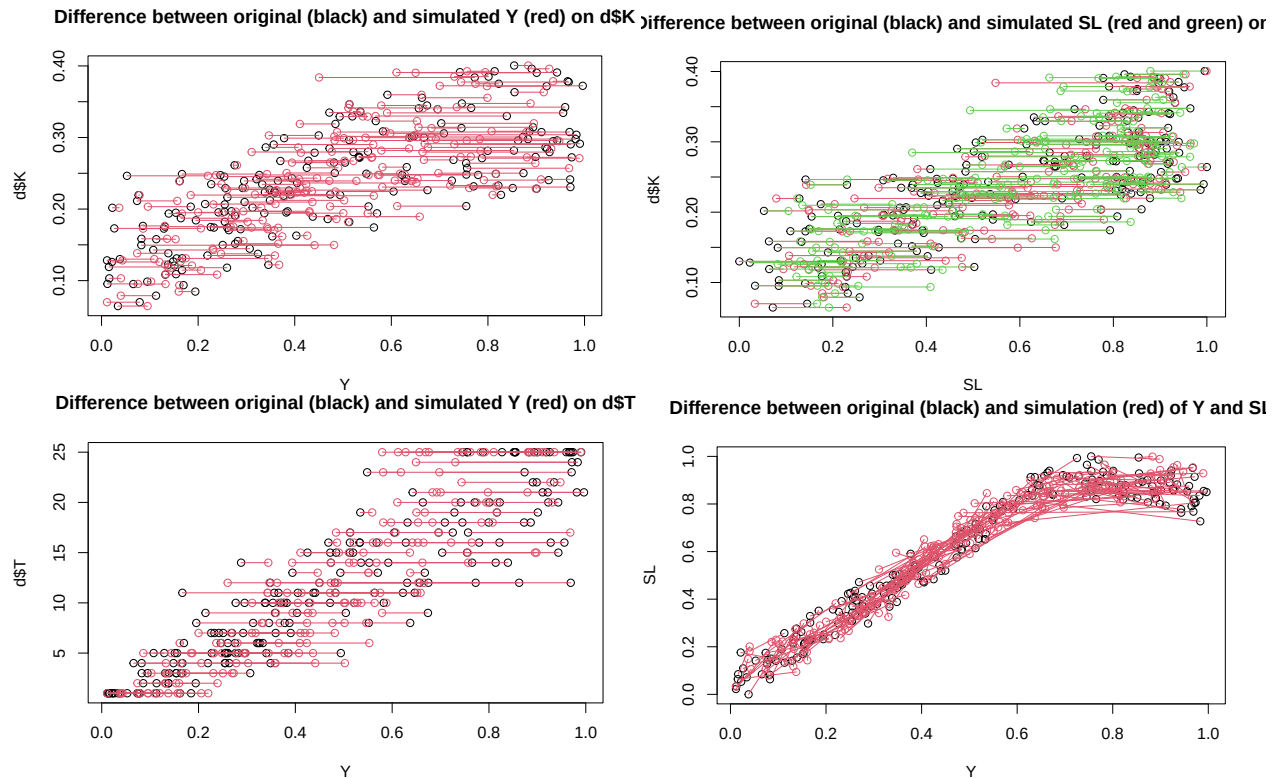
# check Y and SL with K
plot(Y, d$K, main = "Difference between original (black) and simulated Y (red) on d$K")
points(post$Y[1,], d$K, col = 2)
for(i in 1 : 200) lines(c(post$Y[1,i], Y[i]), c(d$K[i], d$K[i]), col = 2)

plot(SL, d$K, main = "Difference between original (black) and simulated SL (red and green) on d$K")
points(post$SL[1,], d$K, col = 2)
for(i in 1 : 200) lines(c(post$SL[1,i], SL[i]), c(d$K[i], d$K[i]), col = 2)
points(post$SL[2,], d$K, col = 3)
for(i in 1 : 200) lines(c(post$SL[2,i], SL[i]), c(d$K[i], d$K[i]), col = 3)

# Y and T
plot(Y, d$T, main = "Difference between original (black) and simulated Y (red) on d$T")
points(post$Y[1,], d$T, col = 2)
for (i in 1:200) lines(c(Y[i], post$Y[1,i]), c(d$T[i], d$T[i]), col = 2)

# Y and SL both hidden parameters
plot(Y, SL, main = "Difference between original (black) and simulation (red) of Y and SL")
points(post$Y[1,], post$SL[1,], col=2)
for(i in 1 : 200) lines(c(Y[i], post$Y[1,i]), c(SL[i], post$SL[1,i]), col = 2)
```





=> this shows that the difference is not very large and hidden parameters have similar trend as the original (synthesized) data.

Posterior Simulation

```
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
Sseq<-seq(0, 0.5, len = 20)
KbyS<- apply(Sseq, # Tricky but t and t to calculate estimates by group, though dK does not have group
  function(i) inv_logit(post$aK + post$bK_S * i + t(t(post$bK_SL) * colMeans(post$SL)) + rep(post$bK,
means<- apply( KbyS , 2 , mean )
PIs<- apply( KbyS , 2 , PI )
lines(Sseq, means, col = 2, lwd = 3)
lines(Sseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Sseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
SLseq<-seq(0, 1, len = 20)
KbySL<- apply(SLseq,
  function(i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + post$bK_SL * i + rep(post$bK_dK, 25) * d$S)
means<- apply( KbySL , 2 , mean )
PIs<- apply( KbySL , 2 , PI )
lines(SLseq, means, col = 2, lwd = 3)
lines(SLseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(SLseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

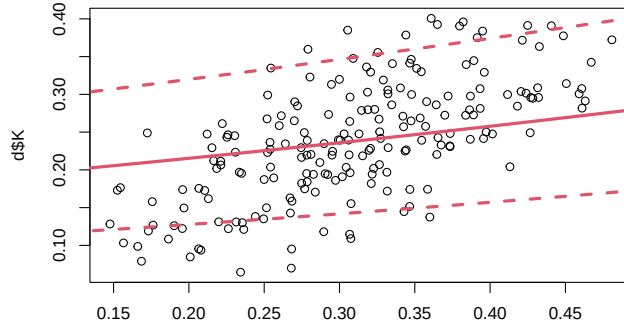
plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
```

```

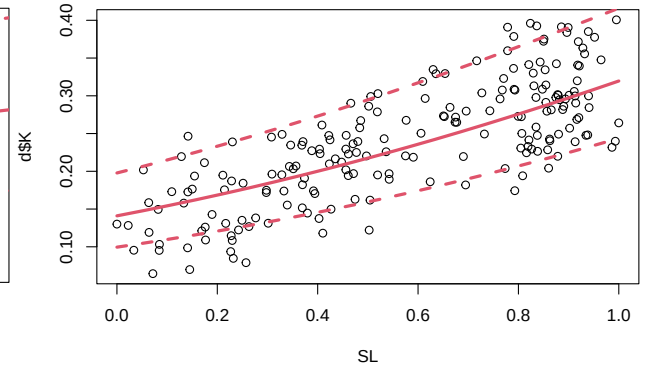
dKseq<-seq(-2.5, 2, len = 20)
KbydK<- sapply(dKseq,
  function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + t(t(post$bK_SL) * colMeans(post$SL)) + rep
PIs<- apply( KbydK , 2 , PI )
means<- apply( KbydK , 2 , mean )
lines(dKseq, means, col = 2, lwd = 3)
lines(dKseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dKseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

```

Synthetic Data Plot(d\$S and d\$K)

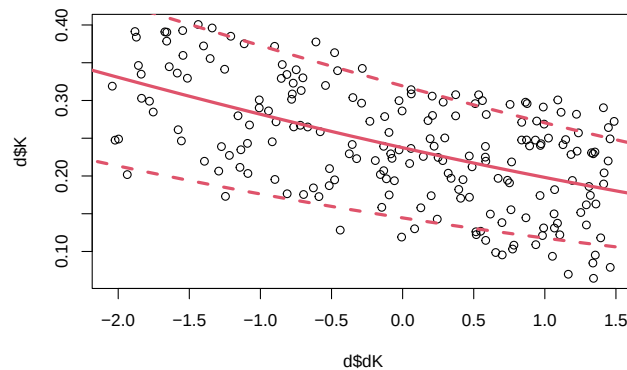


Synthetic Data Plot(SL and d\$K)



Model K

Synthetic Data Plot(d\$dK and d\$K)



- Looks nice!

Model S

- Do the same thing for S
- By the way, there is no post-S exists as it seems absolutely same as post-SL
- would be because it is one of the predictors that i provided in the data.

```
post$SL[1,] == post$S[1,]
```

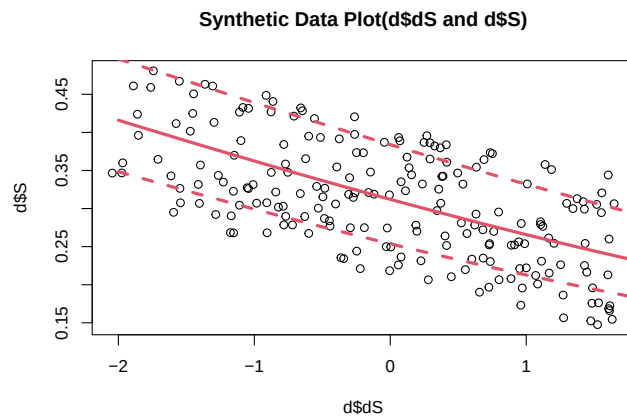
```

## [1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [16] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [31] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [46] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [61] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [76] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [91] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [106] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [121] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [136] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE

```

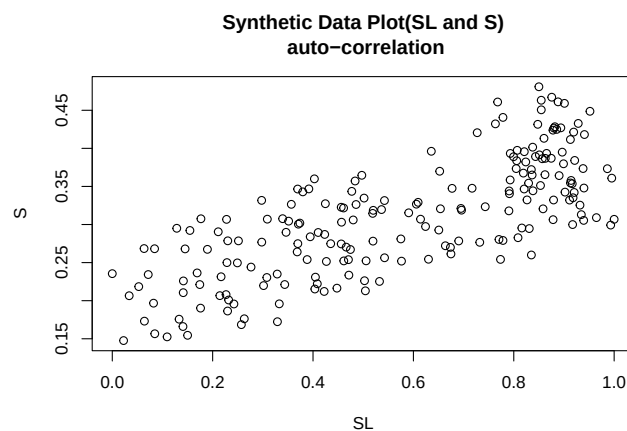
```
## [151] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [166] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [181] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [196] TRUE TRUE TRUE TRUE TRUE
```

```
plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")
dSseq<-seq(-2, 2, len = 20)
SbydS<- sapply(dSseq,
  function(i) inv_logit(post$aS + rep(post$bS_dS, 25) * i)) # Used mean of each simulated data
PIs<- apply( SbydS , 2 , PI )
means<- apply( SbydS , 2 , mean )
lines(dSseq, means, col = 2, lwd = 3)
lines(dSseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dSseq, PIs[2, ], col = 2, lty = 2, lwd = 3)
```



- Functions catch the trend very well!
- The model 1.3 looks good. All posterior prediction plots shows inclusion of data points inside the model scope.
- but also I am wondering if I use correlation between S and SL instead of modeling Y in K model. This is the m2

```
plot(SL, S, main = "Synthetic Data Plot(SL and S)\n auto-correlation")
```



This is because there is a correlation!!

Model 2 (m2)

- see priors in overview.pdf, but this include correlation between SL and S
- This is based on the Ch 14.3 in Statistical Rethinking as SL and S are not centred

- warnings are annoying but relatively few so I just omit it for m2.2 and m.3.

```
# Model 2
m2<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_Y[T] * Y[i] + bK_dK * dK,

    # We do not know anything about each parameter but think similar for all group.
    # So use partial pooling to regularize estimates.
    # mean should start at 0 and has small variation.

    # In this case (m2), we do include correlation between S and SL due to Y.
    vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
    vector [25]: bK_Y <- bK_Y_bar + sigma_bK_Y * z_bK_Y[T],
    c(aK_bar, bK_Y_bar)~normal(0,0.3),
    c(sigma_aK, sigma_bK_Y) ~ exponential(6),
    z_aK[T]~normal(0,0.1),
    z_bK_Y[T] ~ normal(0,0.1),

    transpars>vector[25]:bK_S <- v[,1],
    transpars>vector[25]:bK_SL <- v[,2],

    transpars>matrix[25, 2]:v <- compose_noncentered(sigmaSSL, RhoSSL, zSSL),
    vector[2]: sigmaSSL ~ exponential(2),
    cholesky_factor_corr[2]:RhoSSL ~ lkj_corr_cholesky(1), # I do not expect very very strong corre
    matrix[2,25]: zSSL ~ normal(0 , 1),

    # don't need regularization
    # We are hoping/believing b_dK is negative, but this is not prior.
    bK_dK ~ normal(0, 0.3),
    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS[T] + bS_Y[T] * Y[i] + bS_dS * dS ,

    # I am hoping/believing b_dS is negative and b_Y_S is positive
    # partial pooling for the same reason for a2 and b_Y_S
    vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
    vector[25]: bS_Y<-bS_Y_bar + sigmabS_Y * zbS_Y[T],
    c(aS_bar, bS_Y_bar) ~ normal(0,0.3),
    c(sigma_aS, sigmabS_Y) ~ exponential(6),
    z_aS[T] ~ normal(0,0.1),
    zbS_Y[T] ~ normal(0,0.1),

    # don't need regularization
    bS_dS ~ normal(0,0.3),
    sigma_S ~ exponential(1),

    # Model SL by Y
    vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),
```

```

# Model T, This should be hard coded based on the previous research
T ~ poisson(Y * 25),

# Model T by Y
vector[200]:Y ~ normal( 0.5, 0.2)
), dat = d, constraints = list(
  SL = "lower=0,upper=1", # SL is between 0 and 1
  Y = "lower=0,upper=1" # Y is between 0 and 1
), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
  adapt_delta = 0.99
)
)

```

```

## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write
## 205 | void write_array(stan::rng_t& rng, std::vector<double>& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpS4aHLM/model-34c44e844639.hpp:1789:3: note: by 'void ulam_cm
## 1789 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write
## 136 | void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpS4aHLM/model-34c44e844639.hpp:1789:3: note: by 'void ulam_cm
## 1789 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_cr
## 96 | inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
## | ~~~~~
## C:/Use
## rs/nobut/AppData/Local/Temp/RtmpS4aHLM/model-34c44e844639.hpp:1828:3: note: by 'T_ ulam_cmds
## 1828 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::log
## 154 | inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpS4aHLM/model-34c44e844639.hpp:1828:3: note: by 'T_ ulam_cmds
## 1828 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log
## 91 | inline double log_prob(Eigen::VectorXd& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpS4aHLM/model-34c44e844639.hpp:1828:3: note: by 'T_ ulam_cmds
## const'
## 1828 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_cr
## 159 | inline math::var log_prob(std::vector<math::var>& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpS4aHLM/model-34c44e844639.hpp:1828:3: note: by 'T_ ulam_cmds
## 1828 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
## | ~~~~~

```

```

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [  0%]   (Warmup)
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 1
## Chain 2 Iteration:    1 / 4000 [  0%]   (Warmup)
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: lkj_corr_cholesky_lpdf: Random variable[2] is 0, but must be positive! (in 'C:/Us
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 2
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: lkj_corr_cholesky_lpdf: Random variable[2] is 0, but must be positive! (in 'C:/Us
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 2
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 2
## Chain 3 Iteration:    1 / 4000 [  0%]   (Warmup)
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: lkj_corr_cholesky_lpdf: Random variable[2] is 0, but must be positive! (in 'C:/Us
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 3
## Chain 4 Iteration:    1 / 4000 [  0%]   (Warmup)
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 4

```

```

## Chain 3 Iteration: 100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration: 100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration: 100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration: 100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 4 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 1 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 3 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 1 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 1 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 2 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 3 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 1 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 2 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 4 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 2 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 3 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 1 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 4 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 2 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 1 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)

```

```

## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)

```



```

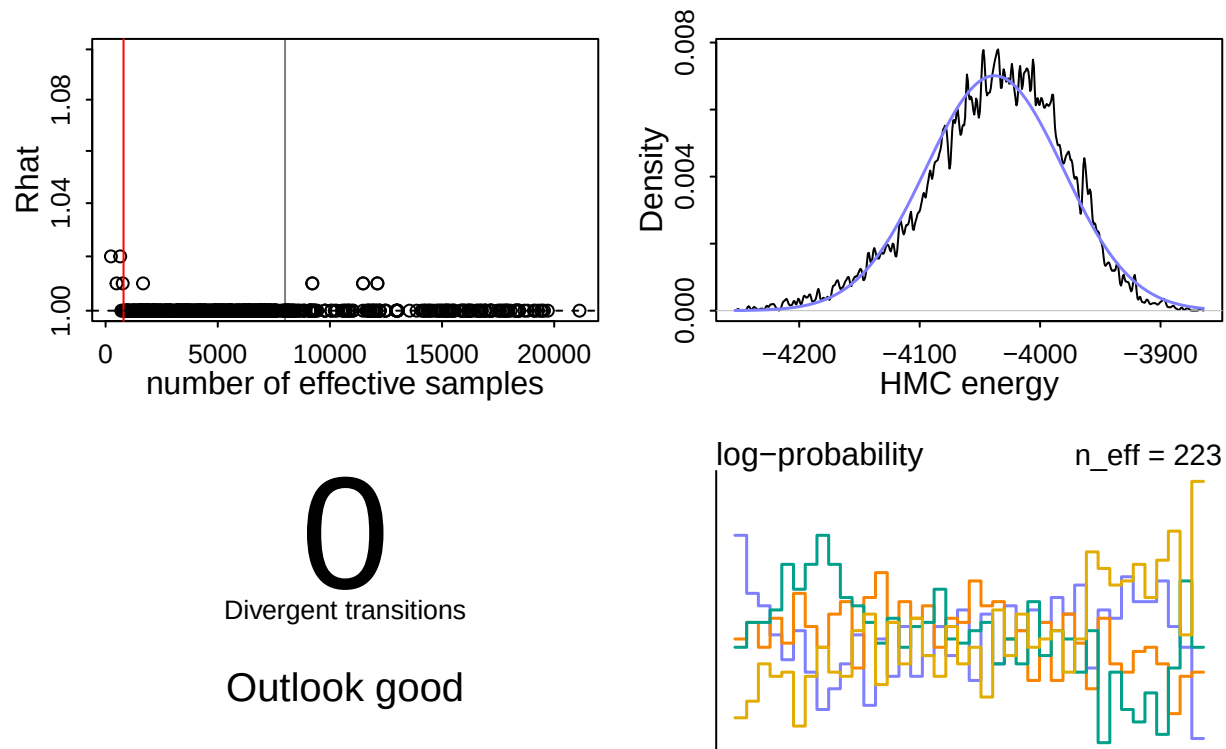
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 119.3 seconds.
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 124.5 seconds.
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)

```

```
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 164.2 seconds.
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 168.2 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 144.0 seconds.
## Total execution time: 168.3 seconds.

## Warning: 4 of 4 chains had an E-BFMI less than 0.3.
## See https://mc-stan.org/misc/warnings for details.
```

dashboard(m2)



-> Looks bad sampling efficiency.

Therefore, I will tackle this problem with the same approach though I do not want to remove the correlation parameters.

```
# Model 2,2
m2.2 <- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_dK * dK,
    vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
    aK_bar ~ normal(0, 0.3),
    sigma_aK ~ exponential(6),
    z_aK[T] ~ normal(0, 0.1),
```

```

transpars>vector[25]:bK_S <- v[,1],
transpars>vector[25]:bK_SL <- v[,2],

transpars>matrix[25, 2]:v <- compose_noncentered(sigmaSSL, RhoSSL, zSSL),
vector[2]: sigmaSSL ~ exponential(2),
cholesky_factor_corr[2]:RhoSSL ~ lkj_corr_cholesky(1),
matrix[2,25]: zSSL ~ normal(0 , 1),
bK_dK ~ normal(0, 0.3),
U ~ exponential(1),

# Model S
S ~ normal(mu_S, sigma_S),
logit(mu_S) <- aS[T] + bS_Y[T] * Y[i] + bS_dS * dS ,
vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
vector[25]: bS_Y<-bS_Y_bar + sigmabS_Y * zbS_Y[T],
c(aS_bar, bS_Y_bar) ~ normal(0,0.3),
c(sigma_aS, sigmabS_Y) ~ exponential(6),
z_aS[T] ~ normal(0,0.1),
zbS_Y[T] ~ normal(0,0.1),
bS_dS ~ normal(0,0.3),
sigma_S ~ exponential(1),

# Model SL by Y
vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

# Model T
T ~ poisson(Y * 25),

# Model T by Y
vector[200]:Y ~ normal( 0.5, 0.2)
), dat = d, constraints = list(
  SL = "lower=0,upper=1",
  Y = "lower=0,upper=1"
), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_tredepth = 13, control = list(
  adapt_delta = 0.99
)
)

```

```
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
```

```
##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:  300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:  200 / 4000 [ 5%] (Warmup)

```

```

## Chain 1 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 4 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 4 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 1 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 1 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 4 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 3 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 2 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 4 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 2 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 1 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 4 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 1 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 4 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 1 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)

```

```

## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)

```

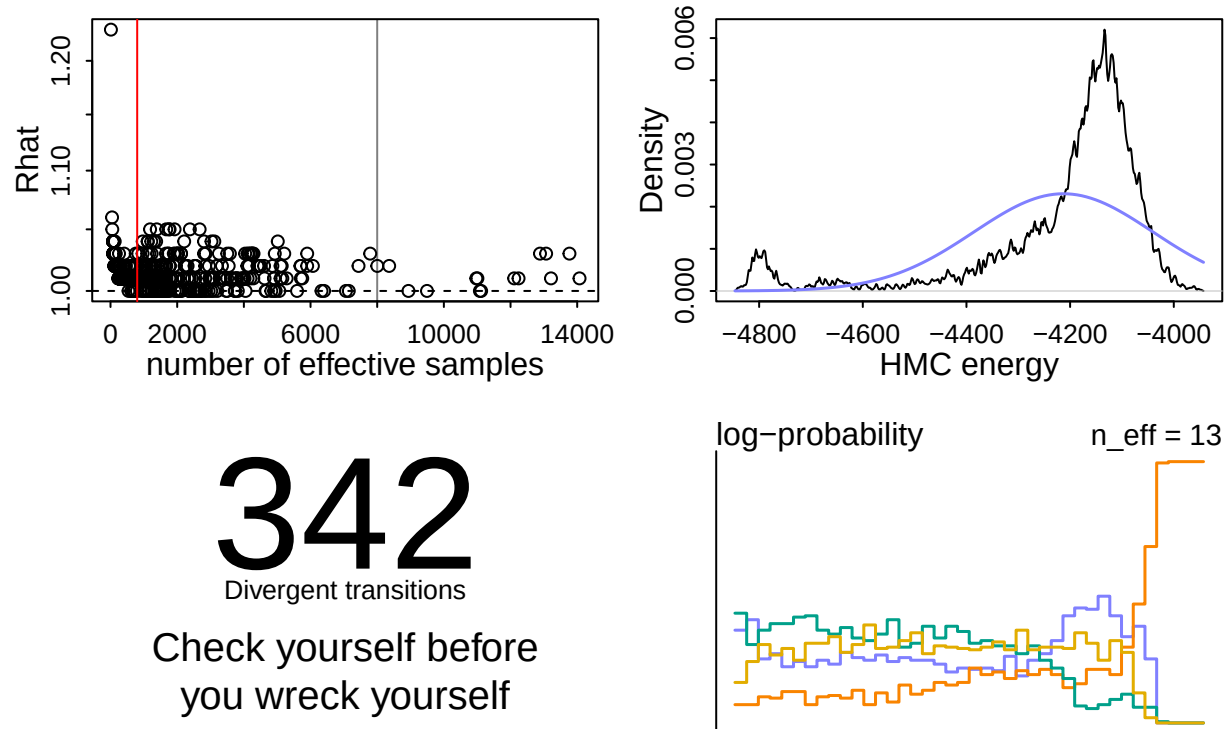
```

## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 208.3 seconds.
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 finished in 212.6 seconds.
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 301.3 seconds.
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 334.8 seconds.
##
## All 4 chains finished successfully.

```

```
## Mean chain execution time: 264.2 seconds.
## Total execution time: 334.9 seconds.
```

```
dashboard(m2.2)
```



- This sampling efficiency is still not great. Let me remove one more thing so that it corresponds m1.3.

```
# Model 2.3
m2.3<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_dK * dK,
    vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
    aK_bar~normal(0,0.3),
    sigma_aK ~ exponential(6),
    z_aK[T]~normal(0,0.1),
    transpars>vector[25]:bK_S <- v[,1],
    transpars>vector[25]:bK_SL <- v[,2],
    transpars>matrix[25, 2]:v <- compose_noncentered(sigmaSSL, RhoSSL, zSSL),
    vector[2]: sigmaSSL ~ exponential(2),
    cholesky_factor_corr[2]:RhoSSL ~ lkj_corr_cholesky(1),
    matrix[2,25]: zSSL ~ normal(0 , 1),
    bK_dK ~ normal(0, 0.3),
    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS[T] + bS_dS * dS ,
    vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
    aS_bar ~ normal(0,0.3),
    sigma_aS ~ exponential(6),
```

```

    z_aS[T] ~ normal(0,0.1),
    bS_dS ~ normal(0,0.1),
    sigma_S ~ exponential(1),

    # Model SL by Y
    vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

    # Model T
    T ~ poisson(Y * 25),

    # Model T by Y
    vector[200]:Y ~ normal( 0.5, 0.2)
  ), dat = d, constraints = list(
    SL = "lower=0,upper=1",
    Y = "lower=0,upper=1"
  ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
    adapt_delta = 0.99
  )
)

```

Running MCMC with 4 parallel chains, with 1 thread(s) per chain...

```

##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration:   400 / 4000 [ 10%] (Warmup)
## Chain 1 Iteration:   400 / 4000 [ 10%] (Warmup)
## Chain 2 Iteration:   400 / 4000 [ 10%] (Warmup)
## Chain 4 Iteration:   400 / 4000 [ 10%] (Warmup)
## Chain 3 Iteration:   500 / 4000 [ 12%] (Warmup)
## Chain 1 Iteration:   500 / 4000 [ 12%] (Warmup)
## Chain 2 Iteration:   500 / 4000 [ 12%] (Warmup)
## Chain 4 Iteration:   500 / 4000 [ 12%] (Warmup)
## Chain 3 Iteration:   600 / 4000 [ 15%] (Warmup)
## Chain 1 Iteration:   600 / 4000 [ 15%] (Warmup)
## Chain 4 Iteration:   600 / 4000 [ 15%] (Warmup)
## Chain 2 Iteration:   600 / 4000 [ 15%] (Warmup)
## Chain 3 Iteration:   700 / 4000 [ 17%] (Warmup)
## Chain 1 Iteration:   700 / 4000 [ 17%] (Warmup)

```



```

## Chain 4 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 3 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 1 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)

```

```

## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 85.3 seconds.
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)

```

```

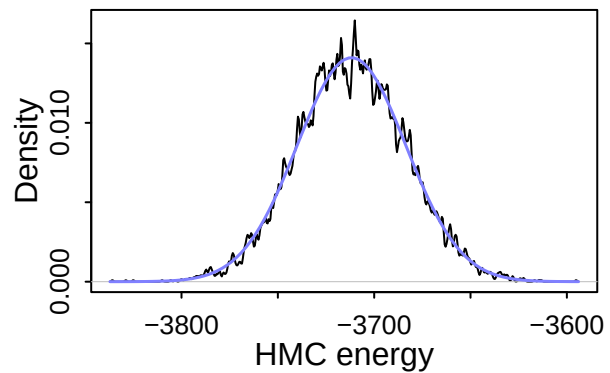
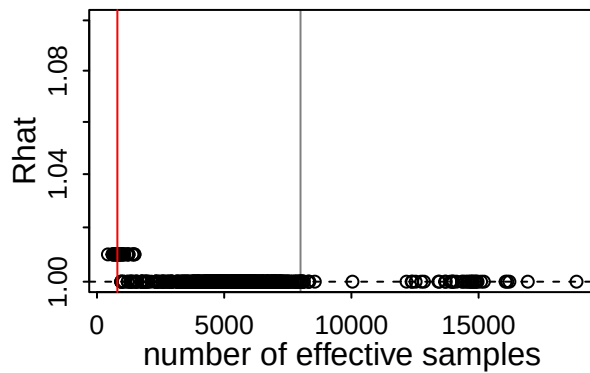
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 117.3 seconds.
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 118.8 seconds.
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 118.9 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 110.1 seconds.
## Total execution time: 119.0 seconds.

```

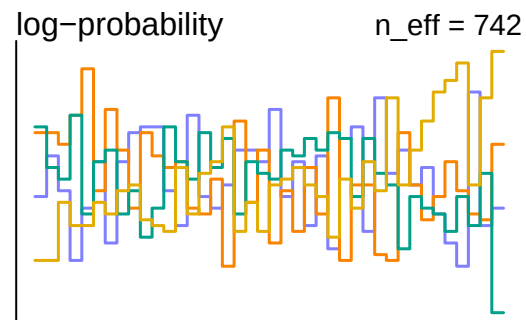
```

dashboard(m2.3)

```



0
Divergent transitions
Outlook good



```
precis(m2.3, depth=1)
```

	mean	sd	5.5%	94.5%	rhat	ess_bulk
aK_bar	-1.47472410	0.076979550	-1.59851787	-1.35700311	1.000530	3260.2549
sigma_aK	0.19733382	0.214307175	0.01131929	0.59494499	1.000481	6877.1763
bK_dK	-0.23055241	0.013341398	-0.25194031	-0.20909377	1.002709	3926.4460
U	0.02958534	0.002539970	0.02526029	0.03339041	1.011857	634.9192
aS_bar	-0.75622966	0.044797274	-0.82570037	-0.68424806	1.000618	2444.9680
sigma_aS	1.67423735	0.244668345	1.31507651	2.09722947	1.000235	3799.6239
bS_dS	-0.22407644	0.013192743	-0.24521917	-0.20301861	1.000371	14769.3257
sigma_S	0.03879084	0.002058858	0.03562580	0.04220290	1.000342	10041.4296

- This model is good enough.

Posterior Check

```
post <- extract.samples(m2.3)
```

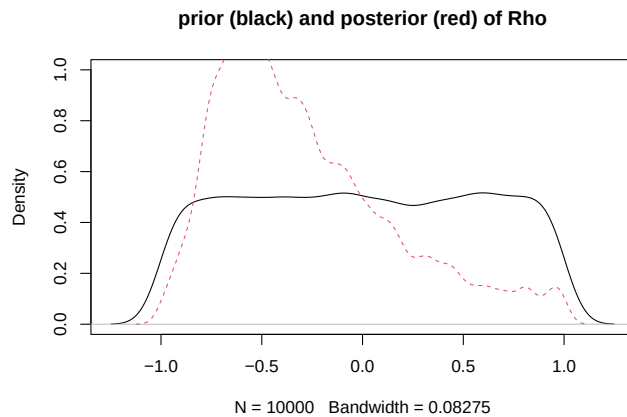
- I do not check the Hidden parameters since I have never touched it since m1

Correlation?

- Rho value shows a bit of correlation between S and SL.

```
R <- rljcorr( 1e4 , K=2 , eta=1 )# eta min lkj_corr()
plot(density( R[,1,2]), ylim = c(0, 1), main = "prior (black) and posterior (red) of Rho")
# This is complicated as cholesky correlation
true_rho_samples=NULL
for (i in 1:2000) {
  L <- post$RhoSSL[i, , ]
  R <- L %*% t(L)
  true_rho_samples[i] <- R[1, 2]
```

```
}
dens(true_rho_samples, add = TRUE, lty = 2, col = 2)
```



Posterior Predictions

```
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
Sseq<-seq(0, 0.5, len = 20)
KbyS<- sapply(Sseq, # Tricky but t and t to calculate estimates by group, though dK does not have group
  function (i) inv_logit(post$aK + post$bK_S * i + t(t(post$bK_SL) * colMeans(post$SL)) + rep(post$bK,
means<- apply( KbyS , 2 , mean )
PIs<- apply( KbyS , 2 , PI )
lines(Sseq, means, col = 2, lwd = 3)
lines(Sseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Sseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
SLseq<-seq(0, 1, len = 20)
KbySL<- sapply(SLseq,
  function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + post$bK_SL * i + rep(post$bK_dK, 25) * d$d
means<- apply( KbySL , 2 , mean )
PIs<- apply( KbySL , 2 , PI )
lines(SLseq, means, col = 2, lwd = 3)
lines(SLseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(SLseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

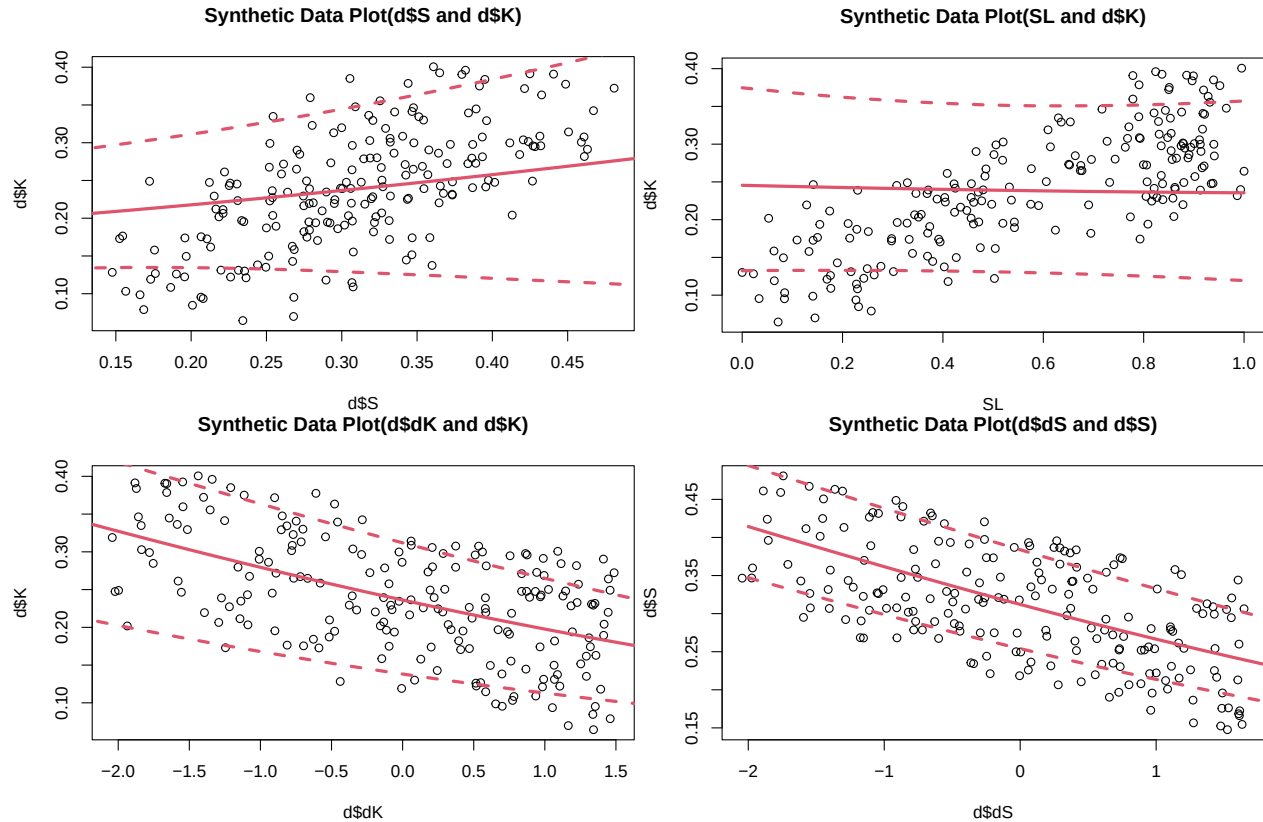
plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
dKseq<-seq(-2.5, 2, len = 20)
KbydK<- sapply(dKseq,
  function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + t(t(post$bK_SL) * colMeans(post$SL)) + rep
PIs<- apply( KbydK , 2 , PI )
means<- apply( KbydK , 2 , mean )
lines(dKseq, means, col = 2, lwd = 3)
lines(dKseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dKseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

# Model S
plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")
dSseq<-seq(-2, 2, len = 20)
SbydS<- sapply(dSseq,
  function (i) inv_logit(post$aS + rep(post$bS_dS, 25) * i )) # Used mean of each simulated data
```

```

PIs<- apply( SbydS , 2 , PI )
means<- apply( SbydS , 2 , mean )
lines(dSseq, means, col = 2, lwd = 3)
lines(dSseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dSseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

```



Overall K estimation is fine, but fit less than m1.3.

Let me compare those two models

Compare Models

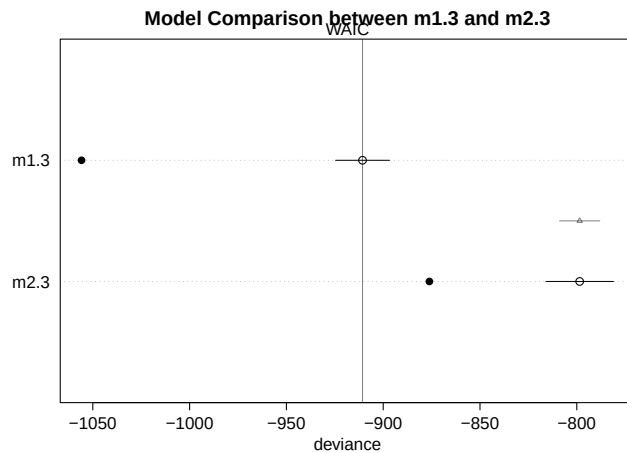
```

# Compare models
model_comparison <- compare(m1.3, m2.3)
print(model_comparison)

##           WAIC          SE    dWAIC      dSE    pWAIC      weight
## m1.3 -910.6769 13.93933   0.0000      NA 72.57796 1.000000e+00
## m2.3 -798.5038 17.47530 112.1731 10.4078 38.81934 4.384585e-25

# Plot WAIC/PSIS differences
plot(model_comparison, main = "Model Comparison between m1.3 and m2.3")

```



Key Columns in the compare Output:

- WAIC / PSIS: Lower values indicate better out-of-sample predictive accuracy.
- dWAIC / dPSIS: The difference in information criteria between the current model and the top-ranked (best) model.
- dSE: If the standard error is large relative to the difference, the two models have statistically indistinguishable predictive capacities.
- weight: The Akaike weight, which represents the relative probability or evidence that a given model will make the best predictions among the set of compared models

=> Therefore, the m1 is better than the other one.

Model 3

- Same thing! See Overview pdf for details.

```
m3<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK + bK_S * S + bK_SL * SL[i] + bK_Y * Y[i] + bK_dK * dK + bK_T[T],

    # Basically same as m1 but not regularization (no pooling).
    # bK_T is regularized to maintain the efficiency.

    c(aK, bK_S, bK_SL, bK_Y, bK_dK) ~ normal(0,0.1), # This is different from the other one, so th
    vector[25]: bK_T <- bK_T_bar + sigma_bK_T * z_bK_T[T],
    bK_T_bar ~ normal(0, 0.1), # regularization but not high sigma
    sigma_bK_T ~ exponential(1), # Needs to be 1 to maintain bK_T.
    z_bK_T[T] ~ normal(0, 0.1),

    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS + bS_dS * dS + bS_Y * Y[i] + bS_T[T],

    # I am hoping/believing b_dS is negative, b_Y_S is positive, and bS_T is different for each one
    c(aS, bS_Y, bS_dS) ~ normal(0,0.1),
    # bS_T is regularized.
```

```

    vector[25]: bS_T <- bS_T_bar + sigma_bS_T * z_bS_T[T],
    bS_T_bar ~ normal(0, 0.1),
    sigma_bS_T ~ exponential(1),
    z_bS_T[T] ~ normal(0, 0.1),

    sigma_S ~ exponential(1),

    # Model SL by Y
    vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

    # Model T
    T ~ poisson(Y * 25),

    # Model T by Y
    vector[200]:Y ~ normal( 0.5, 0.2)
), dat = d, constraints = list(
  SL = "lower=0,upper=1", # SL is between 0 and 1
  Y = "lower=0,upper=1" # Y is between 0 and 1
), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
  adapt_delta = 0.99
)
)

```

```

## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write_
## 205 | void write_array(stan::rng_t& rng, std::vector<double>& theta,
##      | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpS4aHLM/model-34c44c0f22f1.hpp:1308:3: note: by 'void ulam_cm
## 1308 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##      | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write_
## 136 | void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
##      | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpS4aHLM/model-34c44c0f22f1.hpp:1308:3: note: by 'void ulam_cm
## 1308 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##      | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log_
## 91 | inline double log_prob(Eigen::VectorXd& theta,
##      | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpS4aHLM/model-34c44c0f22f1.hpp:1345:3
## : note: by 'T_ ulam_cmdstanr_511c4eb172d16b42fe27f2635eb5e62a_model_namespace::ulam_cmdstanr_511c4
## 1345 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_cr
## 159 | inline math::var log_prob(std::vector<math::var>& theta,
##      | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpS4aHLM/model-34c44c0f22f1.hpp:1345:3: note: by 'T_ ulam_cmds
## 1345 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_crtp
## 96 | inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
##      | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpS4aHLM/model-34c44c0f22f1.hpp:1345:3: note: by 'T_ ulam_cmds

```



```

## pace::ulam_cmdstanr_511c4eb172d16b42fe27f2635eb5e62a_model::log_prob(Eigen::Matrix<T_a, -1, 1>&, std
## 1345 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      |   ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::log
## 154 |   inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
##      |   ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpS4aHLM/model-34c44c0f22f1.hpp:1345:3: note:   by 'T_ ulam_cmds
## 1345 |   log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      |   ~~~~~

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 1
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 2
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 4
## Chain 3 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 1 Iteration:   400 / 4000 [10%] (Warmup)

```

```

## Chain 2 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 4 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 1 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 3 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 3 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 2 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 1 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 4 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 3 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 2 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 4 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 1 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 3 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 2 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 1 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)

```

```

## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)

```

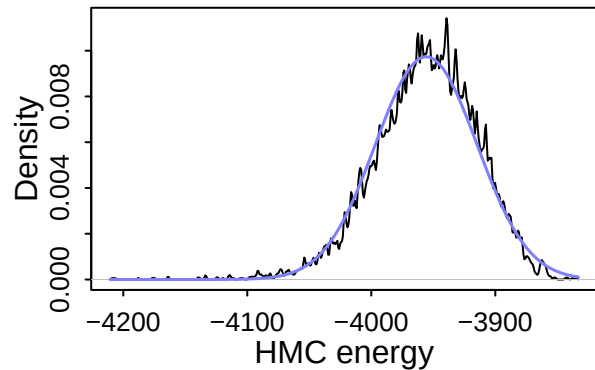
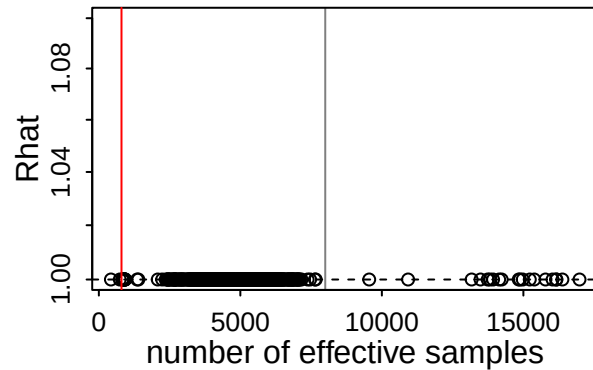
```

## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 188.2 seconds.
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 193.9 seconds.
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 201.6 seconds.
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 254.9 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 209.7 seconds.
## Total execution time: 255.0 seconds.

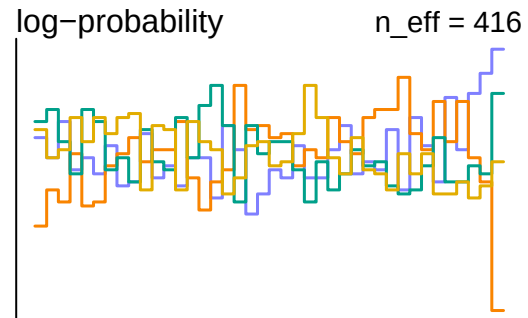
## Warning: 3 of 4 chains had an E-BFMI less than 0.3.
## See https://mc-stan.org/misc/warnings for details.

```

dashboard(m3)



0
Divergent transitions
Outlook good



- without divergent but it has a few problem with efficiency. I will try to eliminate by removing parameters.

```
m3.2 <- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK + bK_S * S + bK_SL * SL[i] + bK_dK * dK + bK_T[T],

    c(aK, bK_S, bK_SL, bK_Y, bK_dK) ~ normal(0, 0.1),
    vector[25]: bK_T <- bK_T_bar + sigma_bK_T * z_bK_T[T],
    bK_T_bar ~ normal(0, 0.1),
    sigma_bK_T ~ exponential(1),
    z_bK_T[T] ~ normal(0, 0.1),

    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS + bS_dS * dS + bS_Y * Y[i] + bS_T[T],

    c(aS, bS_Y, bS_dS) ~ normal(0, 0.1),
    vector[25]: bS_T <- bS_T_bar + sigma_bS_T * z_bS_T[T],
    bS_T_bar ~ normal(0, 0.1),
    sigma_bS_T ~ exponential(1),
    z_bS_T[T] ~ normal(0, 0.1),

    sigma_S ~ exponential(1),

    # Model SL by Y
```

```

        vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

# Model T
    T ~ poisson(Y * 25),

# Model T by Y
    vector[200]:Y ~ normal( 0.5, 0.2)
), dat = d, constraints = list(
    SL = "lower=0,upper=1",
    Y = "lower=0,upper=1"
), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
    adapt_delta = 0.99
)
)

```

Running MCMC with 4 parallel chains, with 1 thread(s) per chain...

```

##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 2 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 4 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 4 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 2 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 3 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 2 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 1 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 3 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 4 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 2 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 3 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 1 Iteration:   900 / 4000 [22%] (Warmup)
## Chain 4 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 2 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 3 Iteration:   800 / 4000 [20%] (Warmup)

```

```

## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 4 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)

```

```

## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)

```



```

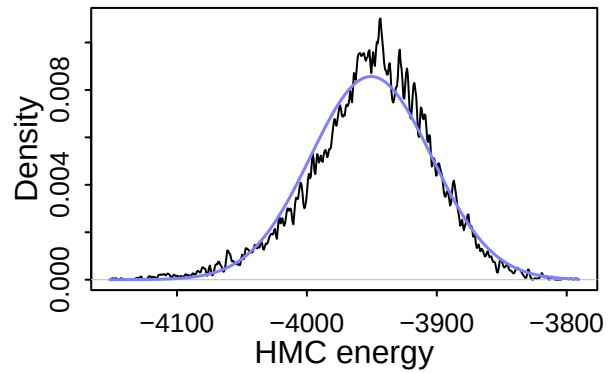
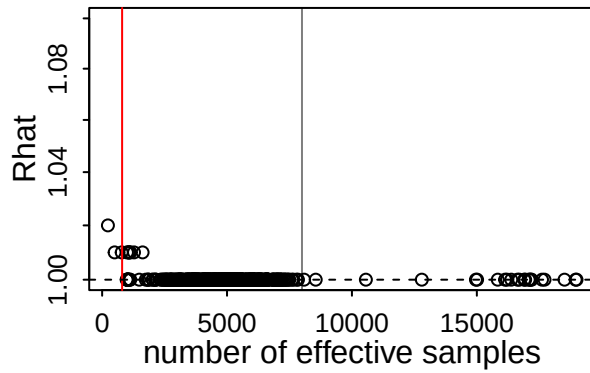
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 26598.6 seconds.
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 26616.0 seconds.
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 26619.3 seconds.
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 26625.5 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 26614.8 seconds.
## Total execution time: 26621.1 seconds.

```

```

dashboard(m3.2)

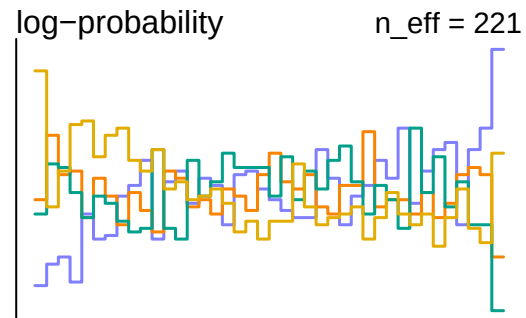
```



0

Divergent transitions

Outlook good



- This is still not the best model with low efficiency. I will remove one more parameter.
- I can remove bK_T or bS_T but this time, I want to make this similar to m1.3 and m2.3.

```
m3.3<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK + bK_S * S + bK_SL * SL[i] + bK_dK * dK + bK_T[T],

    c(aK, bK_S, bK_SL, bK_dK) ~ normal(0,0.1),
    vector[25]: bK_T <- bK_T_bar + sigma_bK_T * z_bK_T[T],
    bK_T_bar ~ normal(0, 0.1),
    sigma_bK_T ~ exponential(1),
    z_bK_T[T] ~ normal(0, 0.1),

    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS + bS_dS * dS + bS_T[T],

    c(aS, bS_dS) ~ normal(0,0.1),
    vector[25]: bS_T <- bS_T_bar + sigma_bS_T * z_bS_T[T],
    bS_T_bar ~ normal(0, 0.1),
    sigma_bS_T ~ exponential(1),
    z_bS_T[T] ~ normal(0, 0.1),

    sigma_S ~ exponential(1),

    # Model SL by Y
    vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),
```

```

# Model T
T ~ poisson(Y * 25),

# Model T by Y
vector[200]:Y ~ normal( 0.5, 0.2)
), dat = d, constraints = list(
  SL  = "lower=0,upper=1",
  Y   = "lower=0,upper=1"
), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
  adapt_delta = 0.99
)
)

```

Running MCMC with 4 parallel chains, with 1 thread(s) per chain...

```

##
## Chain 1 Iteration:      1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:      1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:      1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:      1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:    100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:    100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:    100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:    100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:    200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:    200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration:    200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:    300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration:    300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:    300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration:    200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:    400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:    400 / 4000 [10%] (Warmup)
## Chain 1 Iteration:    400 / 4000 [10%] (Warmup)
## Chain 2 Iteration:    300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration:    500 / 4000 [12%] (Warmup)
## Chain 4 Iteration:    500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:    500 / 4000 [12%] (Warmup)
## Chain 2 Iteration:    400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:    600 / 4000 [15%] (Warmup)
## Chain 4 Iteration:    600 / 4000 [15%] (Warmup)
## Chain 1 Iteration:    600 / 4000 [15%] (Warmup)
## Chain 4 Iteration:    700 / 4000 [17%] (Warmup)
## Chain 3 Iteration:    700 / 4000 [17%] (Warmup)
## Chain 2 Iteration:    500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:    700 / 4000 [17%] (Warmup)
## Chain 4 Iteration:    800 / 4000 [20%] (Warmup)
## Chain 3 Iteration:    800 / 4000 [20%] (Warmup)
## Chain 2 Iteration:    600 / 4000 [15%] (Warmup)
## Chain 1 Iteration:    800 / 4000 [20%] (Warmup)
## Chain 2 Iteration:    700 / 4000 [17%] (Warmup)
## Chain 4 Iteration:    900 / 4000 [22%] (Warmup)
## Chain 1 Iteration:    900 / 4000 [22%] (Warmup)

```

```

## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 2 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)

```

```

## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)

```

```

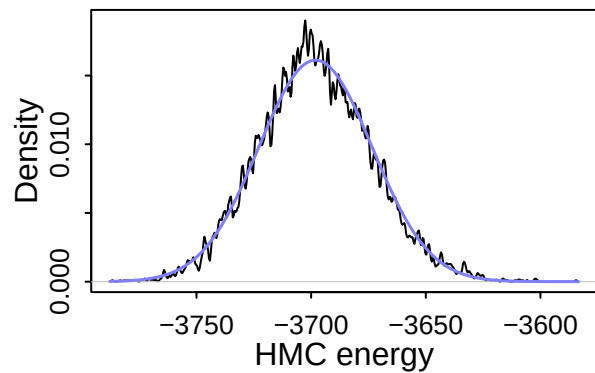
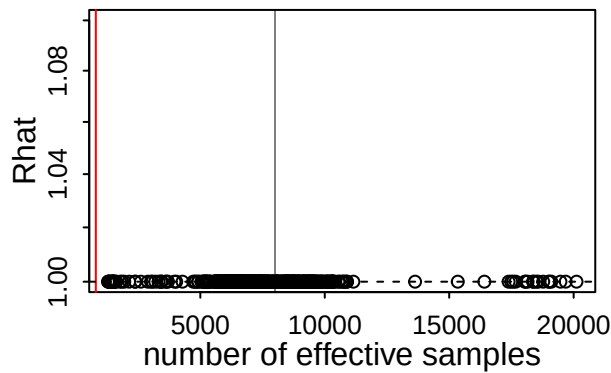
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 96.6 seconds.
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 97.5 seconds.
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 101.9 seconds.
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 127.4 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 105.8 seconds.
## Total execution time: 127.6 seconds.

```

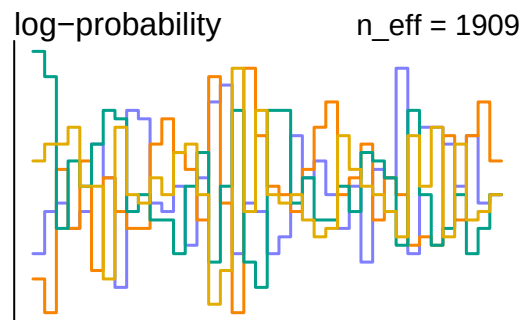
```

dashboard(m3.3)

```



0
Divergent transitions
Outlook good



```
precis(m3.3, depth = 1)
```

	mean	sd	5.5%	94.5%	rhat	ess_bulk
## bK_dK	-0.22916552	0.016471738	-0.25557834	-0.20299173	1.001305	15344.837
## bK_SL	0.41768065	0.098372844	0.25664278	0.57373361	1.002403	2891.204
## bK_S	0.25663164	0.100543631	0.09404386	0.41723121	1.000595	8794.598
## aK	-0.23500520	0.111760556	-0.41709788	-0.05730322	1.001049	4283.767
## bK_T_bar	-0.23316270	0.111860627	-0.41414474	-0.05922329	1.001503	3960.275
## sigma_bK_T	8.53621125	1.815484600	5.82183938	11.52700753	1.003108	1852.219
## U	0.03911559	0.002933690	0.03456186	0.04400546	1.002846	3321.439
## bS_dS	-0.22467918	0.013248391	-0.24568535	-0.20341821	1.001471	18039.287
## aS	-0.31614419	0.081992252	-0.44691326	-0.18335816	1.000180	5306.643
## bS_T_bar	-0.31674616	0.081184863	-0.44346050	-0.18462802	1.000022	5846.811
## sigma_bS_T	2.55331941	0.621171775	1.75003212	3.65939318	1.002577	1943.533
## sigma_S	0.03866036	0.002064746	0.03550786	0.04206117	1.001509	13628.071

Posterior Check

```
post <- extract.samples(m3.3)
```

- Again, I do not check the Hidden parameters since I have never touched it since m1

```
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
Sseq<-seq(0, 0.5, len = 20)
KbyS<- apply(Sseq,
  function(i) inv_logit(post$aK + post$bK_S * i + post$bK_SL * colMeans(post$SL) + post$bK_dK * d$dK)
  means<- apply( KbyS , 2 , mean )
  PIs<- apply( KbyS , 2 , PI )
  lines(Sseq, means, col = 2, lwd = 3)
  lines(Sseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
  lines(Sseq, PIs[2, ], col = 2, lty = 2, lwd = 3)
```

```

plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
points(post$SL[, ], d$K, col = 2) # include the estimated points, though we confirmed above that it is
SLseq<-seq(0, 1, len = 20)
KbySL<- sapply(SLseq,
  function (i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * i + post$bK_dK * d$dK + colMeans(post$
means<- apply( KbySL , 2 , mean )
PIs<- apply( KbySL , 2 , PI )
lines(SLseq, means, col = 2, lwd = 3)
lines(SLseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(SLseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

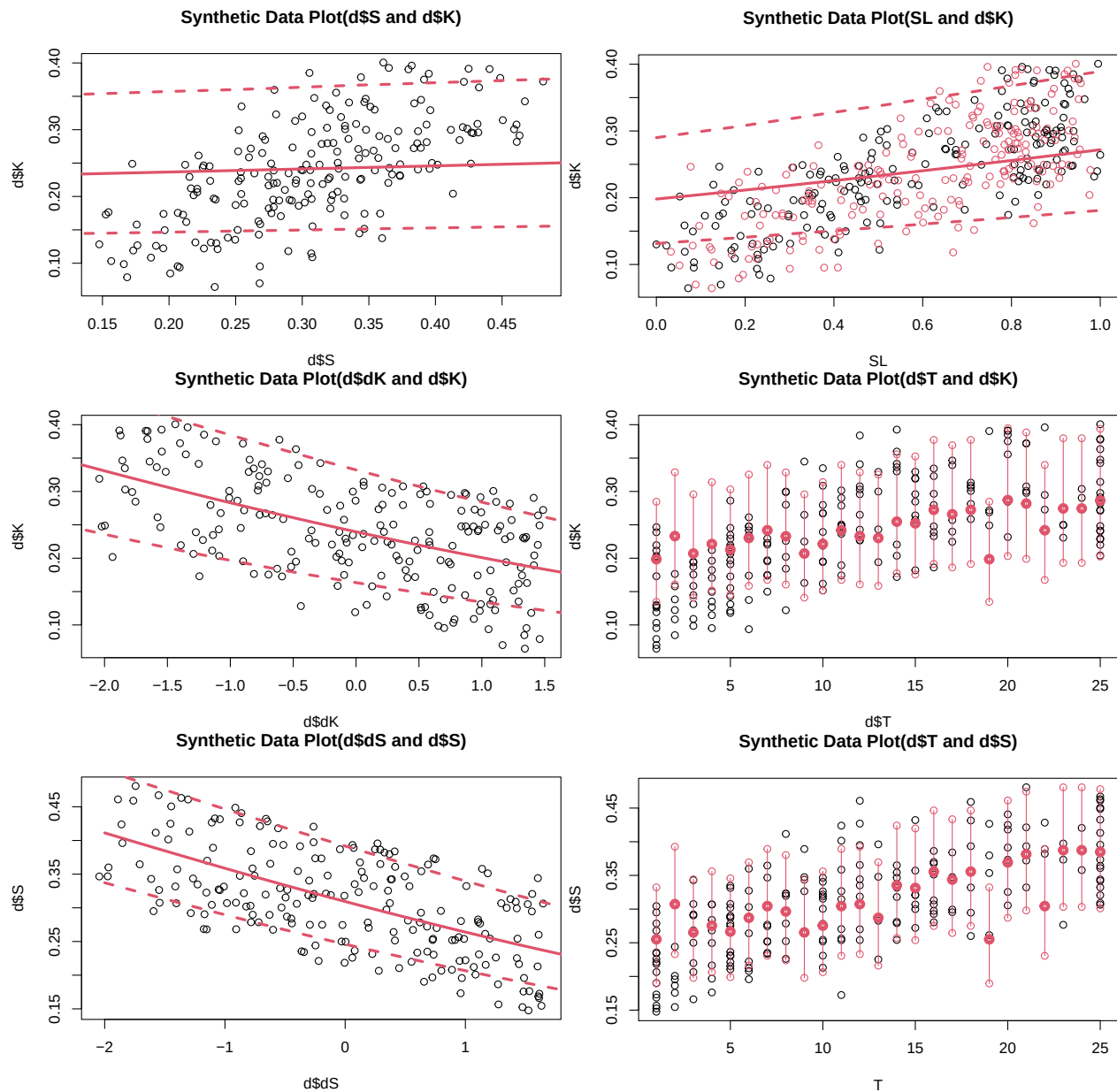
plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
dKseq<-seq(-2.5, 2, len = 20)
KbydK<- sapply(dKseq,
  function (i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * colMeans(post$SL) + post$bK_dK * i +
PIs<- apply( KbydK , 2 , PI )
means<- apply( KbydK , 2 , mean )
lines(dKseq, means, col = 2, lwd = 3)
lines(dKseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dKseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(d$T, d$K, main = "Synthetic Data Plot(d$T and d$K)")
KbyT<- sapply(1:25,
  function (i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * colMeans(post$SL) + post$bK_dK * d$
PIs<- apply( KbyT , 2 , PI )
means<- apply( KbyT , 2 , mean )
points(1:25, means, col = 2, lwd = 4)
points(1:25, PIs[1, ], col = 2, lty = 2 )
points(1:25, PIs[2, ], col = 2, lty = 2 )
for(i in 1:25) lines(c(i,i), c(PIs[1, i], PIs[2, i]), col = 2)

# Model S
plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")
dSseq<-seq(-2, 2, len = 20)
SbydS<- sapply(dSseq,
  function (i) inv_logit(post$aS + post$bS_dS * i + colMeans(post$bS_T)[d$T] )) # Used mean of each s
PIs<- apply( SbydS , 2 , PI )
means<- apply( SbydS , 2 , mean )
lines(dSseq, means, col = 2, lwd = 3)
lines(dSseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dSseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(T, d$S, main = "Synthetic Data Plot(d$T and d$S)")
SbyT<- sapply(1:25,
  function (i) inv_logit(post$aS + post$bS_dS * d$dS + colMeans(post$bS_T)[i]))
PIs<- apply( SbyT , 2 , PI )
means<- apply( SbyT , 2 , mean )
points(1:25, means, col = 2, lwd = 4)
points(1:25, PIs[1, ], col = 2, lty = 2)
points(1:25, PIs[2, ], col = 2, lty = 2)
for(i in 1:25) lines(c(i,i), c(PIs[1, i], PIs[2, i]), col = 2)

```

I have no idea why it does not model bK_T and bS_T very well... I will figure it out. Other than that, looks great, let me compare all those models.

Compare Models

```
# Compare models
model_comparison <- compare(m1.3, m3.3)
print(model_comparison)

##           WAIC          SE    dWAIC      dSE    pWAIC      weight
## m1.3 -910.6769 13.93933   0.0000      NA 72.57796 1.000000e+00
## m3.3 -701.8077 18.16112 208.8691 17.16586 26.10488 4.412123e-46

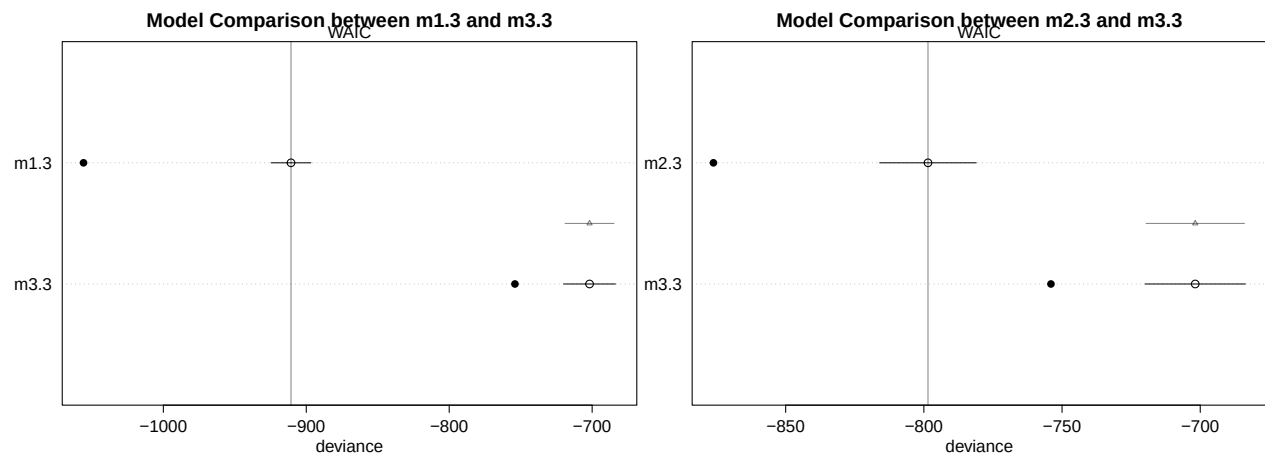
# Plot WAIC/PSIS differences
plot(model_comparison, main = "Model Comparison between m1.3 and m3.3")
```

```
model_comparison <- compare(m2.3, m3.3)
print(model_comparison)
```

```
##           WAIC      SE    dWAIC      dSE    pWAIC      weight
## m2.3 -798.5038 17.47530  0.00000      NA 38.81934 1.000000e+00
## m3.3 -701.8077 18.16112 96.69605 17.80855 26.10488 1.006281e-21
```

```
# Plot WAIC/PSIS differences
```

```
plot(model_comparison, main = "Model Comparison between m2.3 and m3.3")
```



This looks Model 1 is a bit better in this case.

Future Suggestions

- Removing causal relationship in each model (for example, no causal relationship from Y to K) may be examined with actual data.
- The Model 3 still has a problem of bK_T and bS_T as they do not fit the data. I am trying to understand the reason and fix it.

```
rm(list=ls())
```